



**Akademia Górniczo-Hutnicza im. Stanisława Staszica w Krakowie**

Wydział Informatyki

PRACA DYPLOMOWA

**Równoległe i rozproszone metaheurystyki optymalizacyjne  
z desynchronizacją**

**Parallel and distributed optimization metaheuristics with desynchronization**

|                |                                      |
|----------------|--------------------------------------|
| Autor:         | Szymon Żychowicz                     |
| Kierunek:      | Informatyka                          |
| Opiekun pracy: | prof. dr hab. inż. Aleksander Byrski |

Kraków, 2025



*W przygotowaniu pracy magisterskiej wykorzystano następujące narzędzia oparte na sztucznej inteligencji: ChatGPT, Gemini oraz Le Chat. Te wielkie modele językowe (LLM) pomagały redagować tekst (składnia, stylistyka, interpunkcja, synonimy, tłumaczenia, układ w  $\text{\LaTeX}$ ), analizować błędy w kodzie (interpretacja logów i zrzutów błędów), generować kod (szkielety niektórych skryptów służących do tworzenia wykresów oraz uruchamiania algorytmu). Jednakże obliczenia, wnioski, pomysły i zawartość merytoryczna pracy są dziełem autora tekstu.*



## **Streszczenie**

Synchronizacja stanu wiedzy między rozproszonymi węzłami obliczeniowymi w równoległym algorytmie optymalizacyjnym może powodować znaczące opóźnienia w wykonaniu. Wprowadzenie desynchronizacji przyspiesza działanie, jednak może doprowadzić do uzyskania gorszych wyników samego algorytmu. Przedstawiono implementację Evolutionary Multi-agent System z desynchronizacją. Jest to realizacja metaheurystyki optymalizacyjnej, przebadanie efektywności której jest celem tej pracy. Sprawdzono jej wydajność na dużej liczbie węzłów obliczeniowych, wykorzystując do tego infrastrukturę centrum Cyfronet. Wyniki pokazują poprawę w porównaniu do standardowej równoległej implementacji algorytmu.



## **Abstract**

The synchronization of knowledge states among distributed computing nodes in parallel optimization algorithm can cause significant execution delays. Introducing desynchronization accelerates performance, but may also lead to worse algorithm results. This paper presents an implementation of the Evolutionary Multi-agent System with desynchronization, aiming to evaluate the effectiveness of this optimization metaheuristic. Its performance was tested across a large amount of computing nodes using the Cyfronet center's infrastructure. The results show an improvement compared to the standard parallel implementation of the algorithm.



# Spis treści

|                                                             |             |
|-------------------------------------------------------------|-------------|
| <b>Spis rysunków</b>                                        | <b>xi</b>   |
| <b>Spis tabel</b>                                           | <b>xiii</b> |
| <b>Lista kodów źródłowych</b>                               | <b>xv</b>   |
| <b>1. Wstęp</b>                                             | <b>1</b>    |
| <b>2. Optymalizacja i metaheurystyki</b>                    | <b>3</b>    |
| 2.1. Problemy optymalizacyjne . . . . .                     | 3           |
| 2.2. Optymalizacja z wykorzystaniem metaheurystyk . . . . . | 5           |
| 2.3. Zrównoleglenie obliczeń . . . . .                      | 7           |
| 2.4. Wybrane metaheurystyki agentowe . . . . .              | 8           |
| 2.4.1. PSO . . . . .                                        | 8           |
| 2.4.2. ACO . . . . .                                        | 10          |
| <b>3. EMAS</b>                                              | <b>11</b>   |
| 3.1. Historia EMAS . . . . .                                | 11          |
| 3.2. Agenci . . . . .                                       | 12          |
| 3.3. Model wyspowy . . . . .                                | 13          |
| 3.4. Desynchronizacja . . . . .                             | 14          |
| <b>4. Wykorzystane technologie</b>                          | <b>15</b>   |
| 4.1. Frameworki agentowe . . . . .                          | 15          |
| 4.2. Ray framework . . . . .                                | 17          |
| 4.3. SLURM i Ray . . . . .                                  | 18          |
| 4.4. RabbitMQ i AMQP . . . . .                              | 19          |
| <b>5. Implementacja środowiska</b>                          | <b>21</b>   |
| 5.1. Implementacja EMAS . . . . .                           | 21          |
| 5.1.1. Definicja problemu . . . . .                         | 21          |
| 5.1.2. Implementacja agenta . . . . .                       | 23          |
| 5.2. Architektura opóźnień . . . . .                        | 30          |

|                                                                            |           |
|----------------------------------------------------------------------------|-----------|
| <b>6. Badania eksperymentalne</b>                                          | <b>35</b> |
| 6.1. Ograniczenia . . . . .                                                | 35        |
| 6.2. Porównanie algorytmu zdesynchronizowanego i synchronicznego . . . . . | 40        |
| 6.3. Porównanie metod selekcji wysp do emigracji . . . . .                 | 44        |
| 6.3.1. Strategia SendEverywhere . . . . .                                  | 47        |
| 6.3.2. Strategia RandomSelect . . . . .                                    | 48        |
| 6.3.3. Strategia MaxFitnessSelect . . . . .                                | 50        |
| 6.3.4. Strategia MinFitnessSelect . . . . .                                | 51        |
| 6.3.5. Strategia MinStdDevSelect . . . . .                                 | 52        |
| 6.3.6. Strategia MaxDistanceSelect . . . . .                               | 53        |
| 6.3.7. Podsumowanie . . . . .                                              | 54        |
| 6.4. Porównanie sposobów akceptowania imigrantów . . . . .                 | 55        |
| 6.4.1. Strategia accept_everyone . . . . .                                 | 57        |
| 6.4.2. Strategia accept_above_avg . . . . .                                | 57        |
| 6.4.3. Strategia accept_best . . . . .                                     | 58        |
| 6.4.4. Podsumowanie . . . . .                                              | 59        |
| 6.5. Porównanie sposobów wyboru emigrantów . . . . .                       | 61        |
| 6.5.1. Strategia best . . . . .                                            | 62        |
| 6.5.2. Strategia random . . . . .                                          | 62        |
| 6.5.3. Strategia maxDistance . . . . .                                     | 63        |
| 6.5.4. Strategia worst . . . . .                                           | 63        |
| 6.5.5. Podsumowanie . . . . .                                              | 64        |
| <b>7. Zakończenie</b>                                                      | <b>65</b> |
| 7.1. Podsumowanie i wnioski . . . . .                                      | 65        |
| 7.2. Uwagi . . . . .                                                       | 66        |
| 7.3. Dalszy rozwój . . . . .                                               | 67        |
| 7.4. Dodatek – uruchomienie środowiska . . . . .                           | 68        |
| <b>Bibliografia</b>                                                        | <b>69</b> |

# Spis rysunków

|                                                                                                                                                              |    |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------|----|
| 2.1. Problem TSP . . . . .                                                                                                                                   | 4  |
| 2.2. Problem MST . . . . .                                                                                                                                   | 4  |
| 2.3. Wykres 2D funkcji Ackleya . . . . .                                                                                                                     | 4  |
| 2.4. Wykres 3D funkcji Ackleya . . . . .                                                                                                                     | 4  |
| 2.5. Roulette-Wheel Selection . . . . .                                                                                                                      | 6  |
| 2.6. Modele zrównoleglenia metaheurystyk . . . . .                                                                                                           | 7  |
| 2.7. Klatka prezentująca działanie algorytmu PSO z funkcją testową Ackleya . .                                                                               | 9  |
| 2.8. Schemat działania metaheurystyki ACO . . . . .                                                                                                          | 10 |
| 3.1. Schemat działania algorytmu ewolucyjnego . . . . .                                                                                                      | 12 |
| 3.2. Schemat wysp w pełnej topologii . . . . .                                                                                                               | 13 |
| 3.3. Schemat działania zdesynchronizowanego algorytmu EMAS . . . . .                                                                                         | 14 |
| 4.1. Schemat aktorowy platformy Islands Desync . . . . .                                                                                                     | 16 |
| 4.2. Komponenty frameworka Ray . . . . .                                                                                                                     | 17 |
| 4.3. Przykładowy schemat architektury w RabbitMQ . . . . .                                                                                                   | 20 |
| 5.1. Wykres 2D funkcji Rastrigin . . . . .                                                                                                                   | 22 |
| 5.2. Wykres 3D funkcji Rastrigin . . . . .                                                                                                                   | 22 |
| 5.3. Schemat architektury w RabbitMQ dla topologii pierścieniowej z opóźnieniem<br>5ms . . . . .                                                             | 32 |
| 6.1. Ograniczenie RabbitMQ – średnia najlepsza uzyskana wartość fitness w za-<br>leżności od ilości wysp . . . . .                                           | 35 |
| 6.2. Schemat wysp w topologii pierścieniowej . . . . .                                                                                                       | 36 |
| 6.3. Ograniczenie RabbitMQ – czas pobierania osobników . . . . .                                                                                             | 37 |
| 6.4. Ograniczenie RabbitMQ – czas sprawdzenia czy dostępny jest osobnik . . . .                                                                              | 37 |
| 6.5. Ograniczenie RabbitMQ – modele wielomianowych dopasowań do czasów po-<br>bierania imigrantów . . . . .                                                  | 38 |
| 6.6. Ograniczenie RabbitMQ – stosunek czasu wykonywania operacji IO do czasu<br>obsługi zapytania . . . . .                                                  | 39 |
| 6.7. Desynchronizacja, a opóźnienia – porównanie średnich najlepszych wyników<br>algorytmu zdesynchronizowanego i synchronicznego w zależności od opóźnienia | 40 |

|                                                                                                                                                                                                                    |    |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----|
| 6.8. Desynchronizacja, a opóźnienia – porównanie najlepszych wyników algorytmu zdesynchronizowanego i synchronicznego w zależności od opóźnienia z dopasowaniem trendu . . . . .                                   | 41 |
| 6.9. Desynchronizacja, a opóźnienia – porównanie średnich ilości trafień algorytmu zdesynchronizowanego i synchronicznego w zależności od opóźnienia . . . . .                                                     | 42 |
| 6.10. Desynchronizacja, a opóźnienia – porównanie średnich stosunków ilości trafień do ilości przybyłych imigrantów algorytmu zdesynchronizowanego i synchronicznego w zależności od opóźnienia . . . . .          | 43 |
| 6.11. Porównanie średnich najlepszych wyników obliczeń wykorzystujących strategię MaxFitnessSelect w zależności od opóźnienia i współczynnika <i>selection_method_coef</i> . . . . .                               | 46 |
| 6.12. Porównanie najlepszych wyników obliczeń wykorzystujących strategię MaxFitnessSelect w zależności od opóźnienia i współczynnika <i>selection_method_coef</i> z liniami trendu ich średnich . . . . .          | 47 |
| 6.13. Porównanie średnich najlepszych wyników obliczeń wykorzystujących strategię MaxFitnessSelect w zależności od opóźnienia i współczynnika <i>selection_method_coef</i> w zestawieniu z RandomSelect . . . . .  | 50 |
| 6.14. Porównanie średnich najlepszych wyników obliczeń wykorzystujących strategię MinFitnessSelect w zależności od opóźnienia i współczynnika <i>selection_method_coef</i> w zestawieniu z RandomSelect . . . . .  | 51 |
| 6.15. Porównanie średnich najlepszych wyników obliczeń wykorzystujących strategię MinStdDevSelect w zależności od opóźnienia i współczynnika <i>selection_method_coef</i> w zestawieniu z RandomSelect . . . . .   | 52 |
| 6.16. Porównanie średnich najlepszych wyników obliczeń wykorzystujących strategię MaxDistanceSelect w zależności od opóźnienia i współczynnika <i>selection_method_coef</i> w zestawieniu z RandomSelect . . . . . | 53 |
| 7.1. Topologia – siatka 2D z sąsiedztwem Moore’a . . . . .                                                                                                                                                         | 67 |
| 7.2. Topologia – siatka 2D z sąsiedztwem von Neumanna . . . . .                                                                                                                                                    | 67 |

# Spis tabel

|                                                                                                                                                             |    |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------|----|
| 6.1. Wyniki pomiarów działania strategii SendEverywhere oraz ich średnia w zależności od opóźnienia . . . . .                                               | 48 |
| 6.2. Najniższe wartości fitness obliczeń z wykorzystaniem strategii RandomSelect oraz ich średnia w zależności od opóźnienia . . . . .                      | 49 |
| 6.3. Porównanie średnich najlepszych wyników metod SendEverywhere i RandomSelect w zależności od opóźnienia . . . . .                                       | 49 |
| 6.4. Porównanie najniższych wartości fitness uzyskanych bez filtrowania i z eliminacją kopii oraz ich średnie . . . . .                                     | 56 |
| 6.5. Najniższe wartości fitness obliczeń z wykorzystaniem metody <i>accept_everyone</i> oraz ich średnia w zależności od opóźnienia . . . . .               | 57 |
| 6.6. Najniższe wartości fitness obliczeń z wykorzystaniem metody <i>accept_above_avg</i> oraz ich średnia w zależności od opóźnienia . . . . .              | 58 |
| 6.7. Najniższe wartości fitness obliczeń z wykorzystaniem metody <i>accept_best</i> oraz ich średnia w zależności od opóźnienia . . . . .                   | 58 |
| 6.8. Porównanie średnich wyników metod <i>accept_everyone</i> , <i>accept_above_avg</i> oraz <i>accept_best</i> w zależności od opóźnienia . . . . .        | 59 |
| 6.9. Najlepsze wyniki i ich średnie metod <i>accept_everyone</i> and <i>accept_above_avg</i> przy opóźnieniu 25ms . . . . .                                 | 60 |
| 6.10. Najniższe wartości fitness obliczeń z wykorzystaniem metody <i>best</i> oraz ich średnia w zależności od opóźnienia . . . . .                         | 62 |
| 6.11. Najniższe wartości fitness obliczeń z wykorzystaniem metody <i>random</i> oraz ich średnia w zależności od opóźnienia . . . . .                       | 62 |
| 6.12. Najniższe wartości fitness obliczeń z wykorzystaniem metody <i>maxDistance</i> oraz ich średnia w zależności od opóźnienia . . . . .                  | 63 |
| 6.13. Najniższe wartości fitness obliczeń z wykorzystaniem metody <i>worst</i> oraz ich średnia w zależności od opóźnienia . . . . .                        | 63 |
| 6.14. Porównanie średnich najlepszych wyników metod <i>best</i> , <i>random</i> , <i>maxDistance</i> oraz <i>worst</i> w zależności od opóźnienia . . . . . | 64 |



# Lista kodów źródłowych

|                                                                                                                                  |    |
|----------------------------------------------------------------------------------------------------------------------------------|----|
| 5.1. Definicja problemu . . . . .                                                                                                | 21 |
| 5.2. Implementacja $n$ -wymiarowej funkcji testowej Rastrigin (wzór 5.1) rozszerzając klasę Problem (kod źródłowy 5.1) . . . . . | 22 |
| 5.3. Definicja agenta EMAS . . . . .                                                                                             | 23 |
| 5.4. Definicja operatora mutacji . . . . .                                                                                       | 24 |
| 5.5. Definicja funkcji <i>get_truncated_normal</i> . . . . .                                                                     | 24 |
| 5.6. Definicja operatora crossover . . . . .                                                                                     | 25 |
| 5.7. Definicja metody <i>reproduce</i> . . . . .                                                                                 | 26 |
| 5.8. Definicja metody <i>iteration</i> . . . . .                                                                                 | 27 |
| 5.9. Definicja metody <i>remove_dead</i> . . . . .                                                                               | 27 |
| 5.10. Definicja metody <i>select_and_reproduce</i> . . . . .                                                                     | 28 |
| 5.11. Definicja metody <i>fight</i> . . . . .                                                                                    | 29 |
| 5.12. Fragment pliku konfiguracyjnego algorytmu w formacie json (odpowiada architekturze z rysunku 5.3) . . . . .                | 30 |
| 5.13. Skrócony kod źródłowy służący do tworzenia kolejek w RabbitMQ na podstawie konfiguracji . . . . .                          | 31 |
| 5.14. Wycinek kodu źródłowego, za pomocą którego wyspa pobiera imigrantów z kolejki . . . . .                                    | 33 |



# 1. Wstęp

Problemy optymalizacyjne są wszechobecne – od inżynierii, przez informatykę, matematykę, po ekonomię. Podejścia do nich są ekstensywnie opisywane w literaturze naukowej, a ich rozwiązania są stosowane powszechnie w przemyśle [1]. Wraz z kolejnymi wymaganiami rynku i możliwościami technologicznymi, ciągle pojawia się zapotrzebowanie na jeszcze efektywniejsze algorytmy optymalizacyjne.

Jak pokazują rankingi takie jak Top500 [2] czy HPCG [3] z roku na rok komputery HPC są coraz wydajniejsze. Rośnie także ich dostępność. Umożliwia to wykonywanie bardziej złożonych obliczeń, na większej liczbie procesorów.

Niesie to ze sobą możliwości, ale także ryzyka. Przy większej liczbie wykorzystywanych węzłów obliczeniowych popularne algorytmy mogą nie zachowywać się poprawnie. Dodatkowe opóźnienia komunikacyjne czy różnice w obciążeniu węzłów, mogą znacząco spowalniać czas ich wykonania lub powodować małą poprawę w stosunku do przydzielonych zasobów. Wybranie odpowiednich metod komunikacji oraz właściwych punktów synchronizacji w algorytmie jest kluczowe dla dobrego działania równoległych i rozproszonych obliczeń. Problem ten został podjęty w pracy.

W przytoczonym wcześniej badaniu rynku [1] podkreślono znaczenie algorytmów ewolucyjnych wśród metaheurystyk optymalizacyjnych. Procesy ewolucyjne są z natury rozproszone, co sprawia, że bazujące na nich algorytmy pasują do środowisk HPC.

*Evolutionary Multi-agent System*, którego modyfikacja przedstawiona została w pracy jest agentowym podejściem do algorytmów ewolucyjnych. Wykorzystuje abstrakcję agenta do wykonywania kroków ewolucyjnych: selekcji, reprodukcji, mutacji oraz rekombinacji genetycznej (*crossover*). Rywalizują oni ze sobą przeszukując w inteligentny sposób przestrzeń rozwiązań (*exploration & exploitation*). Agenci są zlokalizowani na wyspach, po których migrują, wykorzystując przewagę rozproszonej architektury systemu.

Celem pracy jest przebadanie wpływu desynchronizacji na zaimplementowany algorytm EMAS w środowisku HPC z występującymi opóźnieniami w komunikacji między wyspami na różnych węzłach. Aby to osiągnąć sprawdzone zostały różne podejścia i warianty algorytmu.

Algorytmy dokładne i deterministyczne dla wielu problemów okazują się być niewystarczające. W przypadku problemów NP-trudnych czas ich wykonania rośnie szybciej niż wielomianowo i przy zwiększającej się wielkości problemu stają się niepraktyczne. Algorytmy metaheurystyczne pozwalają w takim przypadku uzyskać dobry wynik w rozsądnym czasie. Jest to ogólna klasa algorytmów łączących eksplorację globalnej i przeszukiwanie lokalnej przestrzeni rozwiązań. Metaheurystyki są jednak bardzo kosztowne obliczeniowo, dlatego ważne są modele współbieżne i rozproszone. Pozwalają one na szybsze wykonanie obliczeń oraz poprawę zachowania różnorodności populacji rozwiązań. Badany w tej pracy algorytm metaheurystyczny EMAS osiąga to dzięki dekompozycji populacji na wyspy.

Ze wzrostem możliwości systemów HPC rośnie potrzeba dostępności różnorodnych rozwiązań do radzenia sobie z problemami optymalizacyjnymi efektywnie wykorzystujących ich architekturę. Jednak wydajne horyzontalne skalowanie ilości wykorzystywanych procesorów jest problemem dla wielu algorytmów. Przyczynami mogą być na przykład niewłaściwe zaplanowanie punktów nielokalnej synchronizacji stanu wiedzy węzłów obliczeniowych lub nieadekwatne mechanizmy komunikacji między komponentami systemu. Zgodnie z ideą *No free lunch theorem* znajdują one również swoje zastosowania, ale potrzebne są także programy radzące sobie z opisanymi problemami. Dlatego w tej pracy podjęte zostało zadanie wybrania algorytmu, który uwzględni w swoim działaniu wspomniane wymagania i ograniczenia.

Obliczenia wykonane zostały na zasobach Cyfronetu [4] poprzez platformę PLGrid [5]. Wykorzystano superkomputer Ares [6], widniejący od 2021. roku na liście Top500 [7]. Algorytm EMAS oraz mechanizm opóźnień został zaimplementowany korzystając z platformy „Islands Desync” [8].

Zawartość pracy:

1. opis implementacji agenta
2. opis wykorzystanej platformy obliczeniowej
3. opis mechanizmu wprowadzania opóźnień w komunikacji
4. pomiary efektywności algorytmu w różnych konfiguracjach i scenariuszach
5. wnioski z badań

## 2. Optymalizacja i metaheurystyki

### 2.1. Problemy optymalizacyjne

Problem optymalizacyjny to problem obliczeniowy polegający na znalezieniu *najlepszego* rozwiązania ze zbioru wszystkich możliwych (zgodnych z zadanymi ograniczeniami) rozwiązań. Robi się to poprzez minimalizację funkcji celu lub kosztu.

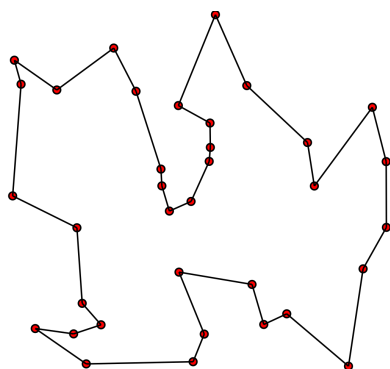
Niech  $M$  to zbiór rozwiązań, a  $f$  to funkcja celu. Oznacza to, że problem optymalizacyjny polega na znalezieniu  $x \in M$ , takiego że:

$$\forall_{y \in M} f(x) \leq f(y) \quad (2.1)$$

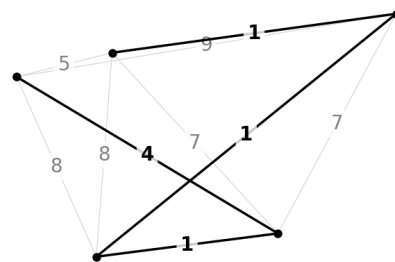
Problemy optymalizacyjne można podzielić na dwa typy. Są to problemy optymalizacyjne dyskretne i ciągłe [9].

Te pierwsze cechuje skończony zbiór rozwiązań  $M$ . Znanymi przykładami są:

- problem komiwojażera (Travelling Salesman Problem – TSP)  
Polega on na znalezieniu minimalnego cyklu Hamiltona w pełnym grafie ważonym. Jako jego ilustrację często podaje się problem przed którym stoi kupiec (komiwojażer), który chce odwiedzić  $n$  miast w taki sposób, że do każdego z nich zawita dokładnie jeden raz (i na koniec powróci do źródłowego), a przebyta droga będzie najkrótsza. Przestrzeń rozwiązań liczy aż  $\frac{(n-1)!}{2}$  kombinacji. [10]
- problem plecakowy (Knapsack Problem)  
Istnieje wiele jego wariantów, ale podstawowy można zdefiniować następująco: dostępnych jest  $n$  przedmiotów o określonej wadze i wartości, a celem jest wybranie takich, których suma wartości jest najwyższa, podczas gdy suma wag pozwala spakować je w plecak. Ilość wszystkich możliwych zbiorów  $n$  przedmiotów wynosi  $2^n$ . [11]
- problem minimalnego drzewa rozpinającego (Minimum Spanning Tree – MST)  
Zadaniem jest wybranie takiego podzbioru krawędzi, które łączą wszystkie wierzchołki (bez cykli) w ważonym grafie nieskierowanym, tak że suma jego wag jest najmniejsza. [12]



Rysunek 2.1.: Problem TSP

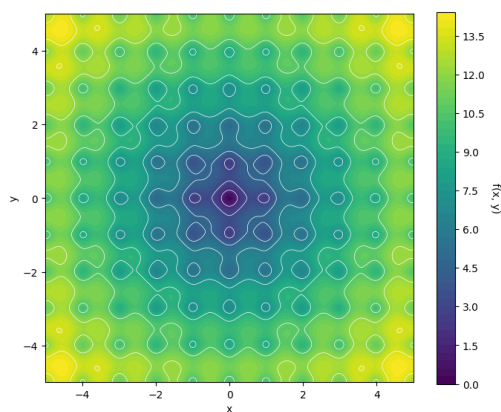


Rysunek 2.2.: Problem MST

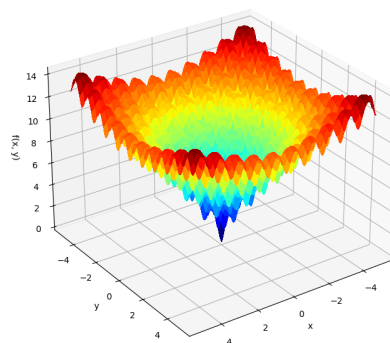
Problemy ciągłe mają natomiast ciągły zbiór rozwiązań. Paradoksalnie może to ułatwić obliczenia, gdyż często znajomość wartości jednego rozwiązania może informować o wartościach w jego otoczeniu. Jako przykłady można podać programowanie liniowe (Linear Programming – LP) lub wybór parametrów do trenowania modelu uczenia maszynowego. Jakość badanego w tej pracy algorytmu weryfikowana jest właśnie poprzez jego efektywność w rozwiązywaniu ciągłych problemów optymalizacyjnych.

Istnieje wiele wykorzystywanych funkcji testowych dla algorytmów rozwiązujących problemy optymalizacyjne ciągłe. Jako przykład można podać funkcję Ackleya, której wzór dla dwóch zmiennych to [13]:

$$f(x, y) = -a \exp \left( -b \sqrt{0.5(x^2 + y^2)} \right) - \exp (0.5(\cos(cx) + \cos(cy))) + a + e \quad (2.2)$$



Rysunek 2.3.: Wykres 2D funkcji Ackleya



Rysunek 2.4.: Wykres 3D funkcji Ackleya

## 2.2. Optymalizacja z wykorzystaniem metaheurystyk

Deterministyczne i dokładne algorytmy źle radzą sobie z problemami optymalizacyjnymi. Wykonują się one zbyt długo. Dlatego skutecznym podejściem jest stosowanie metaheurystyk. Mogą one być zaaplikowane do szerokiej gamy problemów oraz pozwalają uzyskać dobry (choć często nienajlepszy) wynik w rozsądnym czasie [14]. Metaheurystyki dla wielu znanych problemów uzyskują wyniki lepsze niż inne popularne metody [15]. Wśród nich można wymienić Capacitated Vehicle Routing Problem, Open Shop Scheduling, System-on-chip Scheduling, Max-Clique, Minimum Dominating Set czy Subset Sum Problem. Ich wyniki zostały zestawione z benchmarkami, które wykazały jakość podejścia.

Metaheurystyki to wysokopoziomowy (*meta*) sposób szukania rozwiązania (*heurystyka*). Ich zaletą jest niezależność od danego problemu. Stanowią ogólną strategię, którą można dopasować do poszczególnego zagadnienia do rozwiązania. [16]

Metaheurystyki nie są trywialne do sklasyfikowania [17]. Jednak badany w pracy model jest algorytmem ewolucyjnym. Grupa ta inspirowana jest naturą i czerpie z biologii, poprzez wykorzystanie darwinowskiej idei selekcji naturalnej poprzez przekazywanie genów w populacji dla potomków i zachowywanie najlepszych osobników. Wśród niej można wyróżnić między innymi: algorytmy genetyczne, programowanie genetyczne, programowanie ewolucyjne i strategię ewolucyjną. Są to typy algorytmów, które są otwarte na modyfikacje w celu efektywnego dostosowania do danego problemu.

Programowanie genetyczne operuje na populacji programów. Celem jest synteza programu, który dobrze rozwiązuje zadany problem. Osobniki często reprezentowane są w postaci drzewa. Struktura ta pozwala w prosty sposób przeprowadzać na nich mutacje i crossover, które są tutaj kluczowe w przeszukiwaniu przestrzeni rozwiązań. Przykładowe zastosowania tego podejścia to regresja, uczenie bez nadzoru (klasteryzacja) i klasyfikacja. [18]

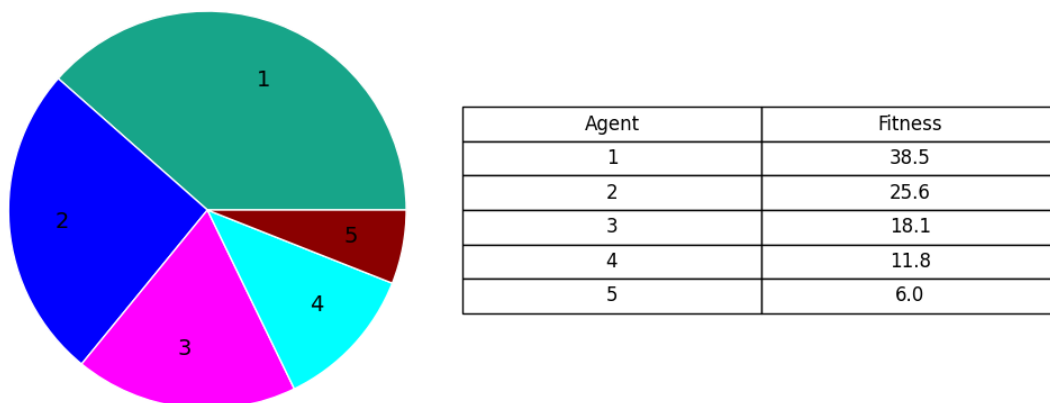
Programowanie ewolucyjne charakteryzuje się aseksualną naturą. Operacja crossover nie występuje w tym typie algorytmu. Jeden rodzic generuje jednego potomka, który podlega mutacji. Jej strategia może zmieniać się między iteracjami i sama adaptować się do danego problemu. [19]

Strategia ewolucyjna także nie wykorzystuje operacji crossover. Różnią się one od programowania ewolucyjnego procesem rywalizacji. W strategii ewolucyjnej jest on deterministyczny i między epokami zachowywane jest  $m$  najlepszych osobników z populacji (w której skład wchodzi  $n$  rodziców i  $n$  powstałych na skutek mutacji potomków). [20, 21]

Badany w pracy algorytm należy do kategorii algorytmów genetycznych. Nowe rozwiązania uzyskiwane są przez rekombinację genetyczną genotypów rodziców, a następnie poprzez ich modyfikację na skutek mutacji. Lepsze rozwiązania mają większe szanse na zachowanie się w populacji, dzięki poczęciu potomstwa, podczas gdy gorsze wymierają. [22]

Do wyboru dostępnych jest wiele metod implementacji operatorów ewolucyjnych algorytmu [23]. Przykładowo:

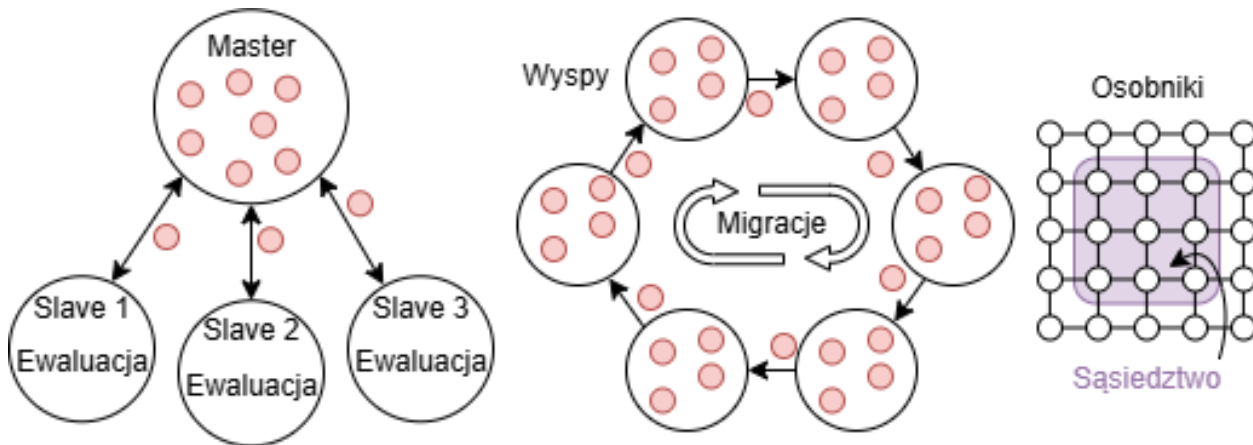
- mutacja: wybór prawdopodobieństwa mutacji danego genu lub funkcji przypisującej mu nową wartość
- selekcja: sposób wyboru rodziców do reprodukcji (np. Roulette-Wheel Selection widoczny na rysunku 2.5)
- crossover: dobranie współczynnika jak duża część genotypu zostanie zaczerpnięta od lepszego rodzica podczas rekombinacji



Rysunek 2.5.: Roulette-Wheel Selection – osobnik z lepszą wartością dostosowania do środowiska (fitness) ma proporcjonalnie większą szansę na zostanie wybranym do selekcji

## 2.3. Zrównoleglenie obliczeń

Algorytmy dokładne napotykają problemy, gdy przestrzeń rozwiązań jest za duża, aby uzyskać wynik w rozsądnym czasie. Można sobie w takich sytuacjach poradzić zrównoleglając metaheurystyki. W przypadku metaheurystyk populacyjnych (takich jak opisane w poprzednim rozdziale) można to zrobić na kilka sprawdzonych sposobów. Popularnymi podejściami są: master-slave, wyspowe, komórkowe. [24]



Rysunek 2.6.: Schemat modeli (kolejno): master-slave, wyspowego i komórkowego

W modelu master-slave centralny procesor odpowiedzialny jest za operację selekcji. Deleguje on osobniki do podległych mu procesorów, które zajmują się ewaluacją rozwiązania i ich zmienianiem (wariacją). Jest to proste podejście, które jednak wymaga dużo synchronizacji.

Najpopularniejszym rozwiązaniem jest podział populacji w przestrzeni i przydzielenie procesorów stosując podejście rozproszone (coarse-grained) lub komórkowe (fine-grained).

W przypadku modelu wyspowego na procesorze wykonywane są wyizolowane kolejne kroki algorytmu. Co wybrany czas następuje migracja osobników na inną wyspę. Wprowadza to dywersyfikację populacji i zapobiega utykaniu algorytmu w minimach lokalnych.

W modelu komórkowym osobniki wchodzi w interakcję tylko ze swoimi sąsiadami w siatce. Nachodzące na siebie sąsiedztwa pozwalają powoli eksplorować przestrzeń analogicznie do zjawiska dyfuzji, a na poziomie sąsiedztwa następuje przeszukiwanie lokalne.

## 2.4. Wybrane metaheurystyki agentowe

Wśród metaheurystyk można wyróżnić metaheurystyki agentowe. Poszukują one rozwiązań problemów, opierając się na interakcjach pomiędzy autonomicznymi jednostkami – agentami. Z prostych oddziaływań pomiędzy nimi emergentnie wyłaniają się złożone zachowania grupowe. Algorytmy oparte na agentach można stosunkowo łatwo zrównoleglic dzięki ich autonomii. [25]

PSO i ACO jako przykłady metaheurystyk agentowych przybliżone zostały poniżej, a badany w pracy EMAS, który także należy do tej grupy, dogłębniej opisany jest w rozdziale 3.

### 2.4.1. PSO

Particle Swarm Optimization (PSO) jest metaheurystyką agentową inspirowaną naturą. W oryginalnej pracy [26] autorzy zaproponowali analogię do stada ptaków, w którym każdy osobnik reprezentuje agenta w populacji. Agenci modyfikują wektory swojej prędkości, wykorzystując zarówno własne doświadczenia, jak i wiedzę grupową, w celu poszukiwania pożywienia.

Niech symbole użyte we wzorach 2.3 i 2.4 oznaczają:

1.  $t$  – numer aktualnego kroku algorytmu
2.  $i$  – numer agenta
3.  $pbest$  – zapamiętaną pozycję najlepszej wartości znalezionej przez agenta
4.  $gbest$  – pozycję globalnie najlepszej wartości znalezionej przez populację
5.  $P$  – aktualną pozycję agenta
6.  $\omega \in [0, 1]$  – „wagę bezwładności”, czyli parametr równoważący eksplorację globalnej i eksploatację lokalnej przestrzeni rozwiązań
7.  $c_1, c_2 \in [0, 1]$  – stałe „współczynniki przyspieszenia”
8.  $r_1, r_2 \in [0, 1]$  – współczynniki losowe wyznaczone w każdej iteracji

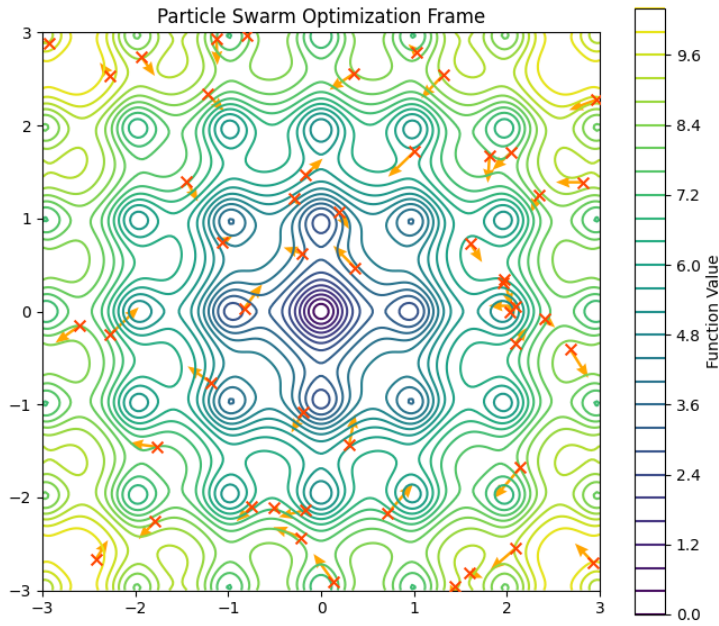
$$V_i(t+1) = \omega V_i(t) + c_1 r_1 (pbest(i, t) - P_i(t)) + c_2 r_2 (gbest(i, t) - P_i(t)) \quad (2.3)$$

$$P_i(t+1) = P_i(t) + V_i(t+1) \quad (2.4)$$

Podstawowa wersja algorytmu po losowym rozmieszczeniu agentów w przestrzeni rozwiązań oraz inicjalizacji zmiennych iteracyjnie wykonuje następujące kroki, aż do spełnienia przyjętego warunku stopu:

1. oblicz nową prędkość  $V_i(t+1)$  na podstawie wzoru 2.3
2. wyznacz nową pozycję  $P_i(t+1)$  agenta zgodnie ze wzorem 2.4
3. oblicz wartość funkcji kosztu w nowej pozycji  $f(P_i(t+1))$ ; jeżeli jest ona lepsza niż dotychczasowa najlepsza  $f(pbest(i,t))$ , wówczas zaktualizuj  $pbest(i,t+1) = P_i(t+1)$  i postępuj analogicznie dla  $gbest$
4. zwiększ wartość  $t$  o jeden

Po zakończeniu pętli wartość  $f(gbest(t))$  stanowi wynik działania algorytmu. Grafika 2.7 przedstawia jego działanie podczas optymalizacji funkcji Ackleya dwóch zmiennych (wzór 2.2). Wybrane parametry to:  $a = 20, b = 0.2, c = 2\pi$ . Symbole  $x$  to agenci, a zaczepione w nich wektory to ich prędkości.

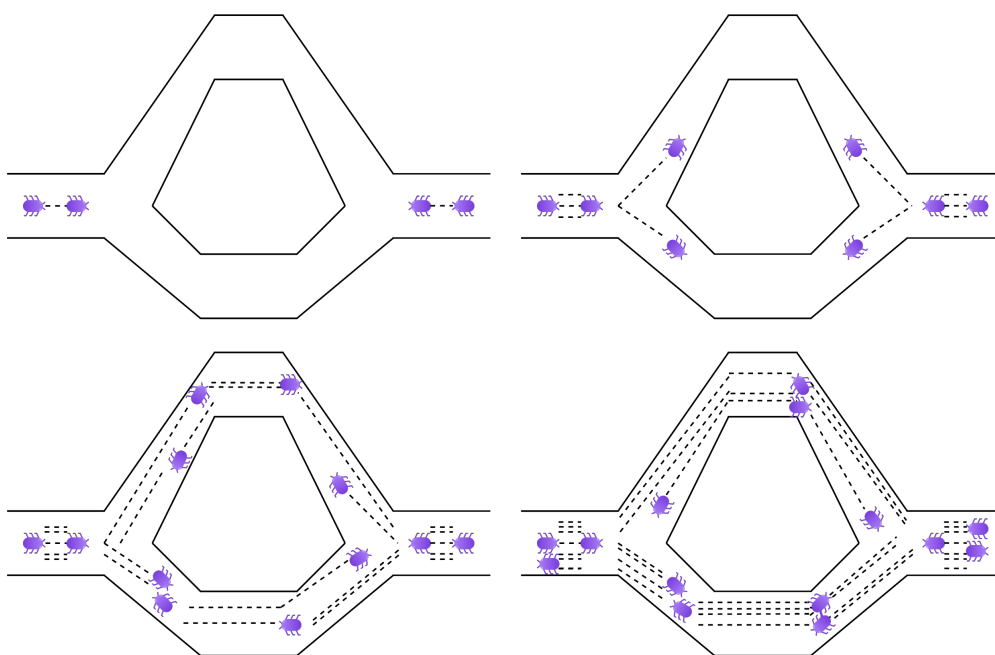


Rysunek 2.7.: Klatka prezentująca działanie algorytmu PSO z funkcją testową Ackleya

### 2.4.2. ACO

Inną popularną metaheurystyką agentową jest Ant Colony Optimization (ACO). Jak sama nazwa wskazuje, algorytm ten inspirowany jest zachowaniem kolonii mrówek. W celu komunikacji owady te wykorzystują pozostawiany za sobą ślad feromonowy. Mechanizm ten pozwala im, bez użycia wskazówek wizualnych, znaleźć najkrótszą drogę od mrowiska do pożywienia.

Proces ten przebiega następująco [27]: gdy pojedyncza mrówka dociera do rozwidlenia dróg, nie mając żadnych wstępnych informacji, wybiera dalszą ścieżkę w sposób losowy. Przy założeniu, że owady poruszają się ze stałą prędkością, trasa, która jest fizycznie krótsza, będzie pokonywana szybciej. W rezultacie mrówki będą nią wędrować częściej, intensyfikując na niej stężenie feromonów. Kolejne owady docierające do tego samego punktu decyzyjnego z większym prawdopodobieństwem podążą za silniejszym śladem. Z czasem krótsza droga ma coraz mocniejszy ślad feromonowy, co z kolei przyciąga jeszcze więcej owadów, aż wszystkie będą chodziły tylko po jednej trasie. Schemat działania opisanego procesu przedstawiony został na grafice 2.8.



Rysunek 2.8.: Schemat działania metaheurystyki ACO

Mrówki mogą łatwo zostać zamodelowane jako agenci, a algorytm zrównoleglony. Metaheurystyka ta sprawdza się w rozwiązywaniu problemu TSP (grafika 2.1).

## 3. EMAS

### 3.1. Historia EMAS

Rozwój coraz bardziej złożonych rozproszonych systemów oraz ograniczenia dotychczas dostępnych algorytmów genetycznych i ewolucyjnych (takie jak: nadmierna centralizacja czy używanie niewystarczających operatorów ewolucyjnych) doprowadziły w 1996 do powstania koncepcji *Evolutionary Multi-agent System* [28].

W kolejnych latach, wraz z dalszą popularyzacją modelu agentowego, dzięki jego użyteczności w środowiskach rozproszonych i zdecentralizowanych, rozwijano wspomniany wyżej model [29, 30]. Został on skutecznie sprawdzony w różnych scenariuszach. Jego użyteczność można było dostrzec w problemach optymalizacji wielokryterialnej, optymalizacji numerycznej, systemach rozmytych (klasyfikacja danych) czy sieciach neuronowych (time-series prediction). Reprezentacja populacji jako zbioru komunikujących się agentów wykonujących kroki ewolucyjne okazała się innowacyjna, co prowadziło do dalszego rozwoju modelu.

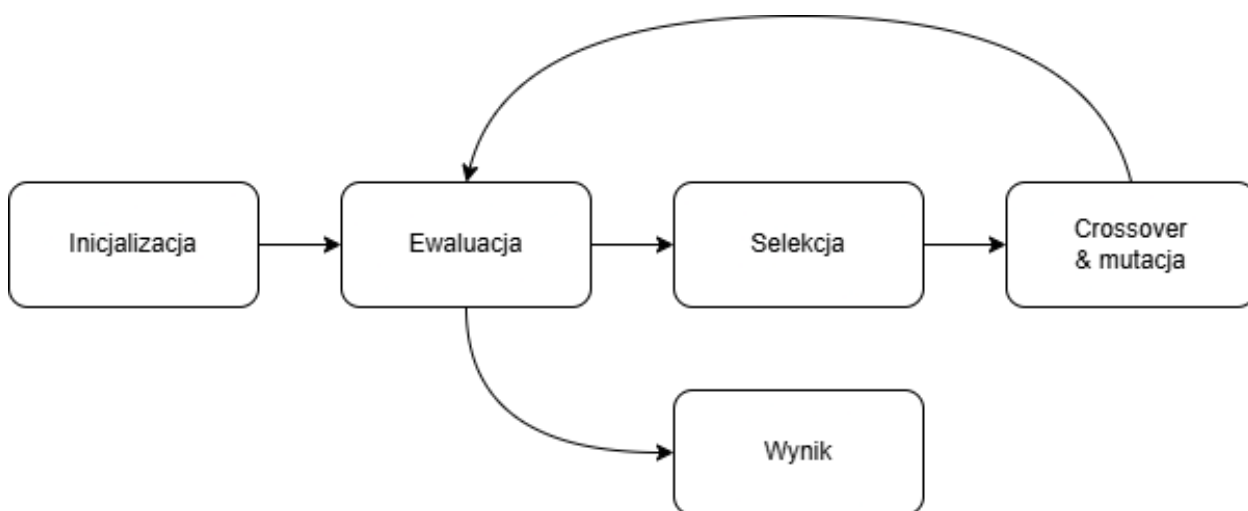
Z czasem powstały nowe warianty [31]. Jako ich przykłady można podać immunologiczny, elitystyczny (*elEMAS*) lub koewolucyjny (*CoEMAS*), które znalazły zastosowania w optymalizacji globalnej, wielokryterialnej lub multimodalnej. Przedstawiono też implementacje do problemów DSS [32], odwrotnych [33], dyskretnych [34] i ciągłych [35]. Pokazuje to, że model jest elastyczny i stosując jego różne warianty oraz modyfikacje można osiągnąć zadowalające rezultaty w szerokiej gamie problemów.

## 3.2. Agenci

Na podstawie dostępnej literatury można sformułować ogólny, intuicyjny opis modelu. Cykl życia agentów najłatwiej zrozumieć poprzez analogię do darwinowskiej idei selekcji naturalnej. Są oni osobnikami populacji (rozwiązaniami). Na podstawie swojego przystosowania do środowiska (jakości rozwiązania problemu) mogą umrzeć, albo przez reprodukcję oddać część swoich genów nowym jednostkom. Decyduje o tym ich *life energy*. Powoduje to, że słabsze rozwiązania są odrzucane, a lepsze promowane i zachowywane. Wprowadzając mutacje można kontrolować różnorodność populacji w celu znajdowania nowych rozwiązań.

Ogólny algorytm można opisać następująco:

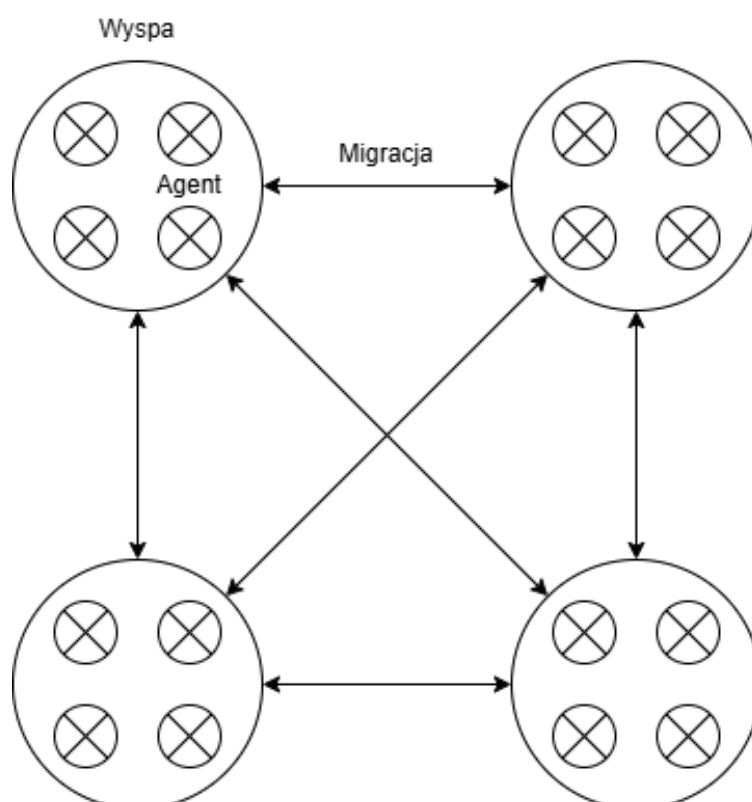
1. wygeneruj populację losowych osobników
2. powtarzaj do osiągnięcia wybranego warunku końca:
  - wyznacz jakość osobników (*evaluation*)
  - wybierz rodziców (*selection*)
  - wygeneruj dzieci przez skrzyżowanie rodziców i mutację potomka (*crossover & mutation*)
  - wyeliminuj słabe osobniki



Rysunek 3.1.: Schemat działania algorytmu ewolucyjnego

### 3.3. Model wyspowy

W EMAS agenci żyją na wyspach, które stanowią między nimi barierę. Opisane wcześniej kroki ewolucyjne odbywają się tylko w kontekście jednej wyspy. Jest to rozwiązanie dostosowane do architektury rozproszonej, do równoległego wykonywania. Po wprowadzeniu operacji migracji agenci mogą przenieść się z jednej wyspy na inną, zgodnie z wybraną topologią.



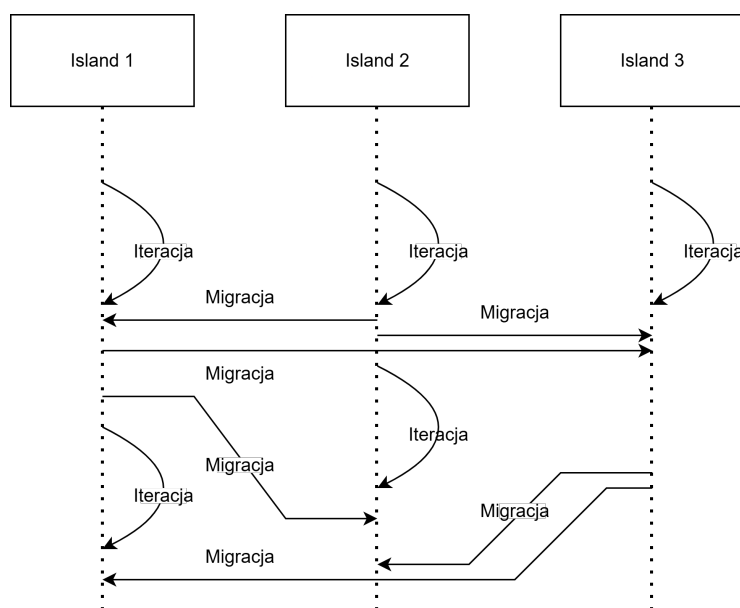
Rysunek 3.2.: Agenci na wyspach w pełnej topologii

W ten sposób dobre rozwiązania są propagowane w systemie bez konieczności ciągłego aktualizowania globalnego stanu wiedzy między wszystkimi węzłami, co wprowadzałoby ogromne opóźnienia w działaniu i nadmiarowy nakład komunikacyjny.

### 3.4. Desynchronizacja

Projektując rozproszony program, który będzie wykonywany na dużej liczbie węzłów obliczeniowych należy uwzględnić powstające ograniczenia komunikacyjne. Aby działał wydajnie trzeba balansować między ilością wysyłanych informacji (przez komunikacyjny *overhead* i czas oczekiwania na informację – *idle time*), a korzyściami, które one przynoszą.

Korzystając z desynchronizacji można osiągnąć znaczne przyspieszenie algorytmu i dzięki temu przeszukać większą część przestrzeni rozwiązań, co może pozwolić uzyskać lepszy wynik [36]. Jednak wprowadza to losowość, która może skutkować uzyskaniem niepoprawnego rezultatu, szczególnie w przypadku symulacji [37]. W przytoczonej pracy sprawdzono różne podejścia do implementacji desynchronizacji w algorytmach optymalizacyjnych i symulacjach. Porównano zaproponowany model zdesynchronizowanego algorytmu Ant Colony Optimization (ACO) [38] z jego klasyczną równoległą wersją. Już zaledwie przy dziesięciu węzłach obliczeniowych można było zobaczyć jego przewagę. W pracy [36] przedstawiono także ideę zdesynchronizowanego algorytmu EMAS widoczną na grafice 3.3 poniżej.



Rysunek 3.3.: Schemat działania zdesynchronizowanego algorytmu EMAS

Brak konieczności synchronizacji wysp po każdej iteracji, a zamiast tego rozsyłanie asynchronicznych wiadomości (migracje) powinno poprawić skalowalność algorytmu.

## 4. Wykorzystane technologie

### 4.1. Frameworki agentowe

Powstało wiele frameworków ułatwiających pracę z agentami. Wśród nich należy wymienić platformy takie jak: MASON [39], RePast [40], MadKit [41] czy AgE [42]. Zapewniają one środowisko do implementacji własnych wieloagentowych rozwiązań. Zarządzają cyklem życia agentów, pozwalają definiować ich akcje i umożliwiają wdrożenie rozproszonej aplikacji [43]. Przytoczone cztery to tylko kilka z otwartoźródłowych frameworków ogólnego zastosowania. Szeroki wybór dostępnych rozwiązań sprawia, że można dopasować odpowiednie do danego problemu.

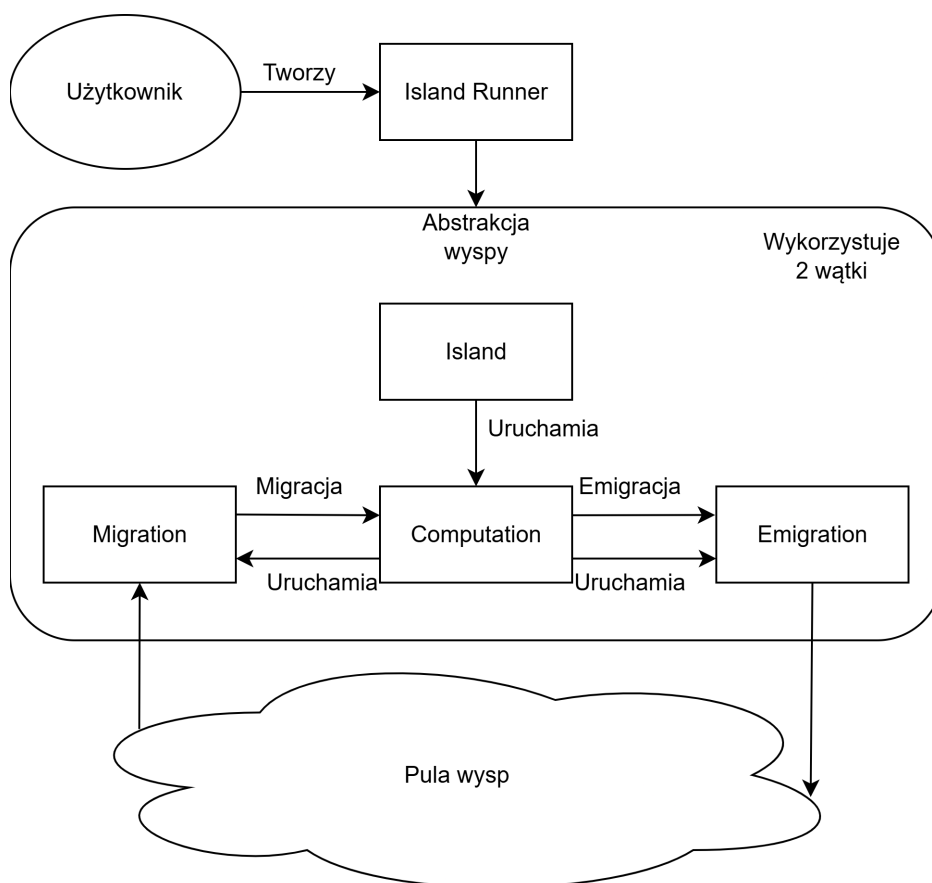
Rozproszenie obliczeń pozwala efektywnie wykonywać duże i złożone zadania, co jest bardzo ważne w dzisiejszych czasach. Framework AgE umożliwia rozszerzyć chmurę obliczeniową, dzięki wykorzystaniu przeglądark na różnych maszynach jako workerów. Oferuje skalowalność bez konieczności zapewnienia kosztownych zasobów [44].

W tej pracy wykorzystana została jednak inna platforma. Jest ściśle dostosowana do środowisk HPC. Opiera się na frameworku Ray w celu zapewnienia lekkiego, rozproszonego środowiska dostępnego w języku Python. [45]

Podstawowymi komponentami platformy są:

- Island – podstawowy aktor przetrzymujący Computation, sąsiednie wyspy w topologii oraz nowych imigrantów
- Computation – aktor zarządzający obliczeniami ewolucyjnymi
- Migration – aktor otrzymujący nowych imigrantów
- Emigration – aktor wysyłający osobników do innych wysp

Ogólny schemat platformy przedstawiony jest na grafice 4.1 z oryginalnej pracy [45]. Będzie ona określana jako „Islands Desync” w dalszych częściach pracy.



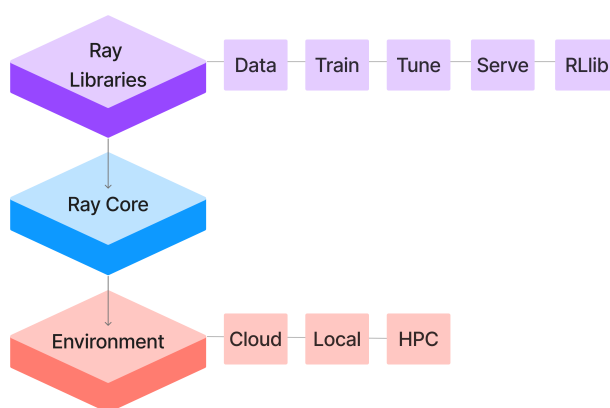
Rysunek 4.1.: Schemat aktorowy platformy „Islands Desync”

## 4.2. Ray framework

Ray [46, 47] to otwartoźródłowy framework do przeprowadzania rozproszonych obliczeń, stworzony dla efektywnego skalowania Pythonowych aplikacji.

Podstawową warstwą platformy jest Ray Core, na której zbudowane są wyższe abstrakcje, takie jak:

- Ray Data – komponent służący do rozproszonego przetwarzania danych. Obsługuje efektywne ładowanie danych oraz ich późniejsze przetwarzanie podczas uczenia, fine-tuningu oraz predykcji w uczeniu maszynowym.
- Ray Train – umożliwia rozproszone trenowanie modeli uczenia maszynowego przy użyciu popularnych bibliotek: PyTorch, TensorFlow, Keras, Horovod, XGBoost, LightGBM. Abstrahuje szczegóły dotyczące dystrybucji danych, synchronizacji wag i komunikacji między workerami.
- Ray Tune – pozwala przeprowadzać tuning parametrów wykorzystując skalowalność systemu. Używa do tego algorytmów takich jak Population Based Training i HyperBand/ASHA oraz zintegrowany jest z wieloma narzędziami do dalszej optymalizacji.
- Ray Serve – komponent do skalowalnego serwowania modeli uczenia maszynowego i endpointów API.
- Ray RLLib – framework do skalowalnego i odpornego na błędy uczenia ze wzmocnieniem. Mimo utrzymania prostego API wykorzystywany jest do nauki różnorodnych aplikacji.



Rysunek 4.2.: Komponenty frameworka Ray

Wracając do Ray Core, na którym oparta jest wykorzystana w pracy platforma „Islands Desync”, wprowadza on trzy typy danych. Są to:

- task – abstrakcja służąca do zrównoleglania Pythonowych funkcji w klastrze
- aktor – pozwala stworzyć workerów utrzymujących wewnętrzny stan między wywołaniami ich metod
- obiekt – Ray zarządza rozproszonym magazynem obiektów (distributed object store) w klastrze

Są one niejednokrotnie wykorzystywane w algorytmie i jego platformie.

### 4.3. SLURM i Ray

Dla systemu z tysiącami procesorów kluczowy jest mechanizm pozwalający efektywnie je wykorzystywać. Simple Linux Utility Resource Management (SLURM) jest menadżerem zasobów w klastrze. Jest wykorzystywany w komputerach Cyfronetu [48]. Kolejkuje on zadania zlecone przez użytkowników, monitoruje stan systemu i zadań, uruchamia aplikacje użytkowników, zarządza konfiguracją maszyny. Jego popularność wśród podobnych rozwiązań dla komputerów HPC wynika z prostoty korzystania dla użytkowników i administratorów, otwartego źródła systemu, przenośności, skalowalności, bezpieczeństwa i wszechstronności. [49]

Uruchamianie środowiska Ray na SLURMie jest – jak przyznają sami twórcy – nieintuicyjne [50]. SLURM wymaga złożenia wielu kopii programu do klastra. Ray natomiast używa jednego head-workera, który przyjmuje i wykonuje program, zarządzając i zlecając zadania wielu innym workerom.

## 4.4. RabbitMQ i AMQP

Ray nie umożliwia sztucznego kontrolowania opóźnień w komunikacji. Ponieważ w pracy badany jest aspekt działania desynchronizacji algorytmu, potrzebny był mechanizm wprowadzenia innych niż normalnie występujące ograniczeń łączności. Z tego powodu zastosowano dodatkową warstwę komunikacyjną pomiędzy wyspami.

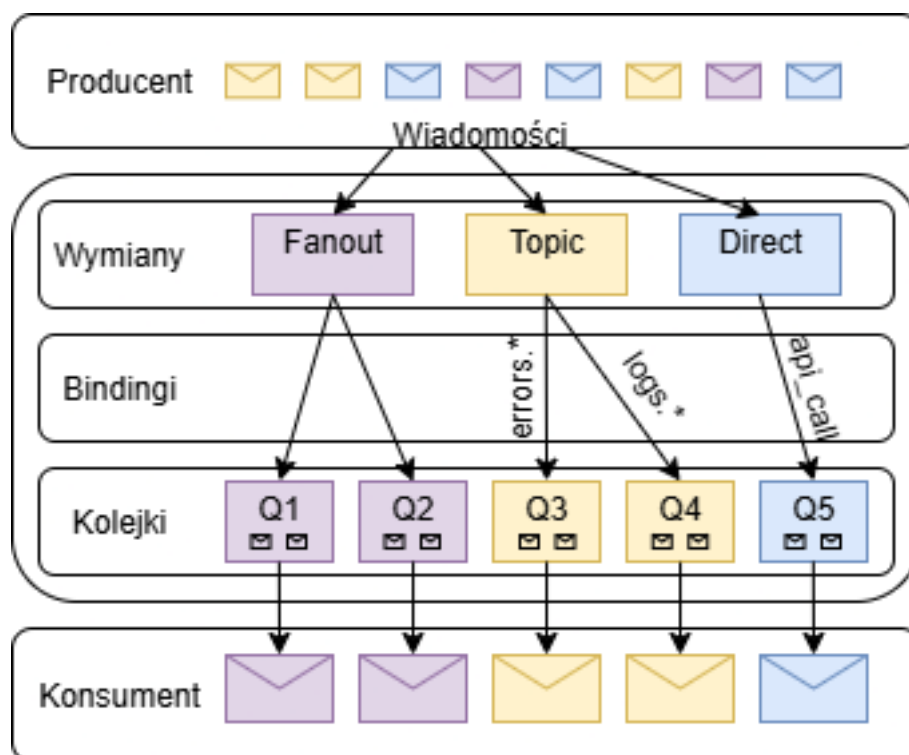
RabbitMQ [51] jest otwartoźródłowym brokerem wiadomości. Jako Message Oriented Middleware typowo jest używany do zapewnienia efektywnej wymiany danych między różnymi, niezależnymi aplikacjami. Korzysta on z protokołu AMQP (Advanced Message Queuing Protocol). [52]

Jest to binarny protokół „poziomu kabla”, który do transmisji wiadomości między aplikacjami wykorzystuje kombinacje store-and-forward, publish-and-subscribe i innych technik. Powstał – jak przyznaje twórca – dzięki doświadczeniom i przez frustracje związane z systemami jakie zastał pracując w bankach inwestycyjnych. Dlatego ważną kwestią, którą adresował protokół są potencjalne straty finansowe, które mogłyby powstać w przypadku zgubienia, spóźnienia lub zniekształcenia wiadomości. [53]

Wiadomości są niemutowalne, mogą być praktycznie dowolnego rozmiaru, a dołączony do nich nagłówek AMQP wykorzystywany jest do podejmowania decyzji o routingu. Ich destynacją są kolejki, które zostawiają dużą dowolność programiście na dostosowanie ich działania do potrzeb aplikacji. Wiadomości są wysyłane do wymiany (Exchange), której konfiguracja decyduje o dalszej drodze. Protokół wymaga wsparcia dla trzech typów wymiany:

1. direct exchange – wiadomość zostanie wysłana do dokładnie jednej kolejki o nazwie zgodnej z kluczem routowania (*routing key*)
2. topic exchange – wiadomość zostanie skopiowana i przekazana wszystkim zainteresowanym kolejkom na podstawie klucza routingu
3. header exchange – wiadomość zostanie skopiowana i przekazania kolejkom, których predykaty zapytania zgodne są z tymi zastanymi w nagłówku

Bindingi to argumenty przekazywane wymianom przez kolejki, na podstawie których podejmowane są decyzje o routingu.



Rysunek 4.3.: Przykładowy schemat architektury w RabbitMQ

Do przeprowadzenia badań potrzebna była kolejka, która pozwala wprowadzić wybrane opóźnienie w przesyłaniu wiadomości. RabbitMQ oferuje mechanizm Dead Letter Exchange.

Wiadomości są ponownie publikowane do wymiany, gdy spełniony zostanie jeden z warunków:

1. wiadomość zostanie negatywnie potwierdzona (*reject* lub *nack*) z parametrem *requeue* ustawionym na fałsz
2. wiadomość wygaśnie przez jej TTL (Time To Live)
3. zostanie przekroczony limit długości kolejki
4. wiadomość zostanie zwrócona do kolejki kworum (replikowalna kolejka używająca algorytmu konsensusu Raft [54] wykorzystywana między innymi także w Kafce czy MongoDB) więcej razy niż limit dostarczeń

Następnie wiadomości przekazywane są dalej do kolejki zgodnie z ich *dead-letter-routing-key*.

# 5. Implementacja środowiska

## 5.1. Implementacja EMAS

### 5.1.1. Definicja problemu

Problem optymalizacyjny reprezentowany jest przez klasę abstrakcyjną (kod źródłowy 5.1), której podstawową metodą jest *evaluate*. Przyjmuje ona jako argument  $x$  rozwiązanie i zwraca jego wartość funkcji celu.

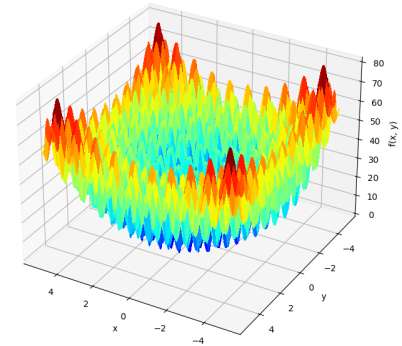
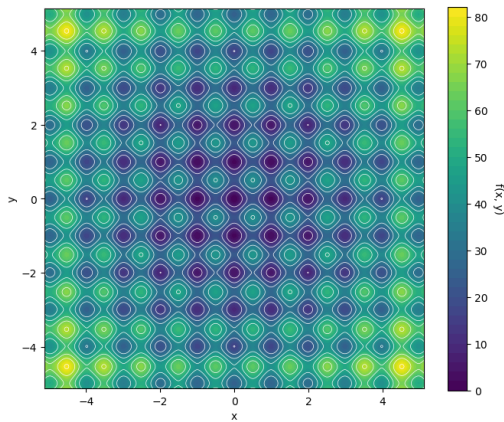
Kod źródłowy 5.1.: Definicja problemu

```
1 class Problem:
2     def __init__(self):
3         raise NotImplementedError("Abstract class")
4
5     def evaluate(self, x):
6         raise NotImplementedError("Abstract method")
7
8     def name(self):
9         raise NotImplementedError("Abstract method")
```

W badaniach wykorzystana jest funkcja testowa Rastrigin (stuwymiarowa). Charakteryzuje się ona dużą liczbą minimów lokalnych, co sprawia, że jej optymalizacja jest trudna. Jej wzór dla  $n$  zmiennych to:

$$f(x) = An + \sum_{i=1}^n [x_i^2 - A \cos(2\pi x_i)] \quad (5.1)$$

Dla  $A = 10$  i  $x \in [-5.12, 5.12]$  funkcja Rastrigin dwóch zmiennych wygląda następująco:



Rysunek 5.1.: Wykres 2D funkcji Rastrigin      Rysunek 5.2.: Wykres 3D funkcji Rastrigin

Kod źródłowy 5.2.: Implementacja  $n$ -wymiarowej funkcji testowej Rastrigin (wzór 5.1) rozszerzając klasę Problem (kod źródłowy 5.1)

```
1 class Rastrigin(Problem):
2     def __init__(self, n_dim, a=10.0):
3         assert n_dim > 0
4         assert a != 0
5
6         self.n_dim = n_dim
7         self.a = a
8
9     def evaluate(self, x):
10        assert len(x) == self.n_dim
11
12        return self.a * len(x) + sum(
13            [x[i]**2 - self.a * cos(2*pi*x[i]) for i in range(len(x))]
14        )
15
16    def name(self):
17        return "RAST"
```

### 5.1.2. Implementacja agenta

Agent zdefiniowany jest jako klasa (kod 5.3) przechowująca:

- genotyp  $x$  w formie tablicy liczb zmiennoprzecinkowych
- energię życiową *energy*
- instancję problemu *problem*
- jakość rozwiązania *fitness*
- dolną i górną wartość (*lower\_bound*, *upper\_bound*) jaką pojedynczy gen może przyjąć

Kod źródłowy 5.3.: Definicja agenta EMAS

```
1 class Agent:
2     def __init__(
3         self,
4         x: list[float],
5         energy: float,
6         problem: Problem,
7         lower_bound: float,
8         upper_bound: float
9     ):
10         self.x = x
11         self.energy = energy
12         self.problem = problem
13         self.fitness = problem.evaluate(x)
14         self.lower_bound = lower_bound
15         self.upper_bound = upper_bound
```

Agent ma zaimplementowane operatory ewolucyjne mutacji i crossoveru, a selekcja odbywa się na poziomie algorytmu, gdyż bierze w niej udział wielu agentów.

Metoda *mutate* (kod źródłowy 5.4) jako argument przyjmuje współczynnik mutacji *mutation\_coef*  $\in [0, 1]$ . Dla każdego genu osobnika przypisuje mu (z prawdopodobieństwem równym współczynnikowi mutacji) wartość zwracaną przez funkcję *get\_truncated\_normal* (kod 5.5), a następnie dokonuje ewaluacji nowego rozwiązania.

Funkcja *get\_truncated\_normal* korzysta z biblioteki Scipy do uzyskania wartości losowej z rozkładu normalnego o średniej *mean* i odchyleniu standardowemu *sd*, przyciętego w *low* i *upp*.

Kod źródłowy 5.4.: Definicja operatora mutacji

```
1 def mutate(self, mutation_coef: float):
2     for i in range(len(self.x)):
3         if random() < mutation_coef:
4             self.x[i] = get_truncated_normal(
5                 mean = self.x[i],
6                 sd = 1,
7                 low = self.lower_bound,
8                 upp = self.upper_bound
9             )
10
11     self.fitness = self.problem.evaluate(self.x)
```

Kod źródłowy 5.5.: Definicja funkcji *get\_truncated\_normal*

```
1 def get_truncated_normal(mean, sd, low, upp):
2     return truncnorm.rvs(
3         (low - mean) / sd,
4         (upp - mean) / sd,
5         loc = mean,
6         scale = sd
7     )
```

Statyczna metoda *crossover* (kod źródłowy 5.6) jako argumenty przyjmuje dwóch agentów *p1* i *p2* (rodziców) oraz zmiennoprzecinkowy współczynnik *split*  $\in [0, 1]$ . Zwraca parę genotypów dzieci powstałych przez skrzyżowanie genotypów rodziców w taki sposób, że prawdopodobieństwo tego, że *i*-ty gen pierwszego dziecka, będzie pochodził od pierwszego rodzica z (a drugiego – od drugiego rodzica) wynosi *split*.

Kod źródłowy 5.6.: Definicja operatora crossover

```
1  @staticmethod
2  def crossover(p1, p2, split: float):
3      c1, c2 = [], []
4
5      for i in range(len(p1.x)):
6          if random() < split:
7              c1.append(p1.x[i])
8              c2.append(p2.x[i])
9          else:
10             c1.append(p2.x[i])
11             c2.append(p1.x[i])
12
13     return c1, c2
```

Metoda *crossover* wywoływana jest w statycznej metodzie *reproduce* (kod źródłowy 5.7), która przyjmuje dwóch agentów *p1* i *p2* (rodziców wybranych wcześniej na drodze selekcji), średnią wartość funkcji celu wszystkich agentów na wyspie *fitness\_average*, początkową liczbę agentów na wyspie *n\_agent*, aktualną liczbę agentów na wyspie *agents\_count*, oraz zmiennoprzecinkowe parametry *energy\_reproduce\_loss\_coef*, *cross\_coef*, *mutation\_coef*  $\in [0, 1]$ .

Rodzice przekazują część swojej energii (*energy\_reproduce\_loss\_coef*) dzieciom, które powstają na podstawie genotypu uzyskanego przez wywołanie metody *crossover*. Następnie jeśli wartość *fitness* osobnika jest lepsza niż *fitness\_average*, to zmniejszana jest szansa mutacji genu o połowę, a w przeciwnym wypadku jest ona podwajana. Po przeprowadzeniu mutacji na potomkach są oni porównywani i jeśli *agents\_count* jest większe niż połowa *n\_agents*, to zwracane jest dziecko o lepszej wartości funkcji celu. Jeśli *agents\_count* jest mniejsze, to zwracana jest para nowych agentów, aby zachować liczbę agentów na wyspie blisko stałego poziomu.

Kod źródłowy 5.7.: Definicja metody *reproduce*

```
1 def reproduce(  
2     p1,  
3     p2,  
4     energy_reproduce_loss_coef: float,  
5     cross_coef: float,  
6     mutation_coef: float,  
7     fitness_average: float,  
8     n_agent: int,  
9     agents_count: int  
10 ):  
11     init_energy1 = p1.energy * energy_reproduce_loss_coef  
12     init_energy2 = p2.energy * energy_reproduce_loss_coef  
13     init_energy = init_energy1 + init_energy2  
14  
15     p1.energy -= init_energy1  
16     p2.energy -= init_energy2  
17  
18     c1_x, c2_x = Agent.crossover(p1, p2, cross_coef)  
19     c1 = Agent(  
20         c1_x, init_energy, p1.problem, p1.lower_bound, p1.upper_bound  
21     )  
22     c2 = Agent(  
23         c2_x, init_energy, p1.problem, p1.lower_bound, p1.upper_bound  
24     )  
25  
26     if c1.fitness < fitness_average: c1.mutate(mutation_coef / 2)  
27     else: c1.mutate(mutation_coef * 2)  
28     if c2.fitness < fitness_average: c2.mutate(mutation_coef / 2)  
29     else: c2.mutate(mutation_coef * 2)  
30  
31     if agents_count < n_agent / 2:  
32         c1.energy /= 2  
33         c2.energy /= 2  
34     return [c1, c2]  
35     return [c1] if c1.fitness < c2.fitness else [c2]
```

Klasa *EMAS* odpowiedzialna jest za przeprowadzanie kolejnych iteracji (kod źródłowy 5.8) algorytmu genetycznego wewnątrz jednej wyspy pomiędzy migracjami. Usuwanie martwych agentów odbywa się w metodzie *remove\_dead* (kod 5.9), selekcja i reprodukcja w metodzie *select\_and\_reproduce* (kod 5.10), a walka osobników w metodzie *fight* (kod 5.11). Poza tym śledzi stan wyspy między iteracjami.

Metoda *select\_and\_reproduce* filtruje agentów na wyspie, aby wybrać takich, którzy mają *energy* powyżej poziomu określonego przez parametr *reproduce\_energy* z konfiguracji algorytmu, a następnie dobiera ich w losowe pary do reprodukcji. Zwraca ona listę nowych potomków.

Metoda *fight* dobiera losowo pary agentów do walki. Jej efektem jest przekazanie przez przegranego osobnika części swojej *energy* (równej współczynnikowi *energy\_fight\_loss\_coef*  $\in [0, 1]$ ). Za pomocą parametru *energy\_diff\_loss\_coef*  $\in [0, 1]$  można odebrać więcej energii, gdy agenci są do siebie podobni (na podstawie podobieństwa genotypu).

Kod źródłowy 5.8.: Definicja metody *iteration*

```
1 def iteration(self, it) -> int:
2     children = self.select_and_reproduce()
3     self.fight()
4     self.remove_dead()
5     self.agents.extend(children)
6     # ...
7     return len(children)
```

Kod źródłowy 5.9.: Definicja metody *remove\_dead*

```
1 def remove_dead(self):
2     self.agents = [
3         agent for agent in self.agents
4         if agent.energy > self.config.alive_energy
5     ]
```

Kod źródłowy 5.10.: Definicja metody *select\_and\_reproduce*

```
1 def select_and_reproduce(self):
2     fitness_average = sum(
3         [agent.fitness for agent in self.agents]
4     ) / len(self.agents)
5     c = []
6     p = list(
7         filter(
8             lambda a: a.energy > self.config.reproduce_energy,
9             self.agents
10        )
11    )
12    shuffle(p)
13    agents_count = len(self.agents)
14    for i in range(len(p) // 2):
15        offspring = Agent.reproduce(
16            p[i],
17            p[len(p) // 2 + i],
18            self.config.energy_reproduce_loss_coef,
19            self.config.cross_coef,
20            self.config.mutation_coef,
21            self.config.alive_energy,
22            fitness_average,
23            self.config.n_agent,
24            agents_count
25        )
26        agents_count += len(offspring)
27        c.extend(offspring)
28
29    return c
```

Kod źródłowy 5.11.: Definicja metody *fight*

```
1 def fight(self):
2     shuffle(self.agents)
3
4     n = len(self.agents) // 2
5     for i in range(n):
6         a1, a2 = self.agents[i], self.agents[n+i]
7         if a1.fitness > a2.fitness:
8             a1, a2 = a2, a1
9
10        energy_loss = a2.energy * self.config.energy_fight_loss_coef
11
12        diff = np.sum(np.abs(np.array(a1.x) - np.array(a2.x)))
13        diff /= (a1.upper_bound - a1.lower_bound)
14        diff /= len(a1.x)
15
16        energy_loss += (a2.energy
17                        * (1-diff) * self.config.energy_diff_loss_coef)
18        if a2.energy - energy_loss < self.config.alive_energy:
19            energy_loss = a2.energy - self.config.alive_energy
20
21        a1.energy += (energy_loss + self.config.alive_energy)
22        a2.energy -= (energy_loss + self.config.alive_energy)
```

Opisane mechanizmy zostały zintegrowane z platformą „Islands Desync”. Dostarcza ona środowisko dla opisanych klas i wprowadza operatory migracji między wyspami i zapewnia rozproszenie oraz zrównoleglenie modelu.

## 5.2. Architektura opóźnień

Platforma „Islands Desync” zapewnia mechanizm migracji, który jednakże nie umożliwia wprowadzenia opóźnień. W celu zbadania efektywności algorytmu zdesynchronizowanego i porównania go z jego synchroniczną wersją potrzebna była gruntowna zmiana sposobu przesyłania informacji między wyspami.

Rysunek 5.3 przedstawia schemat architektury w RabbitMQ dla topologii pierścieniowej z opóźnieniem 5ms. Na jego przykładzie przybliżony poniżej został mechanizm wprowadzenia opóźnień do systemu.

Źródłem prawdy dla architektury kolejek jest pole o nazwie „island\_delays” w pliku konfiguracyjnym widoczne na jego fragmencie 5.12.

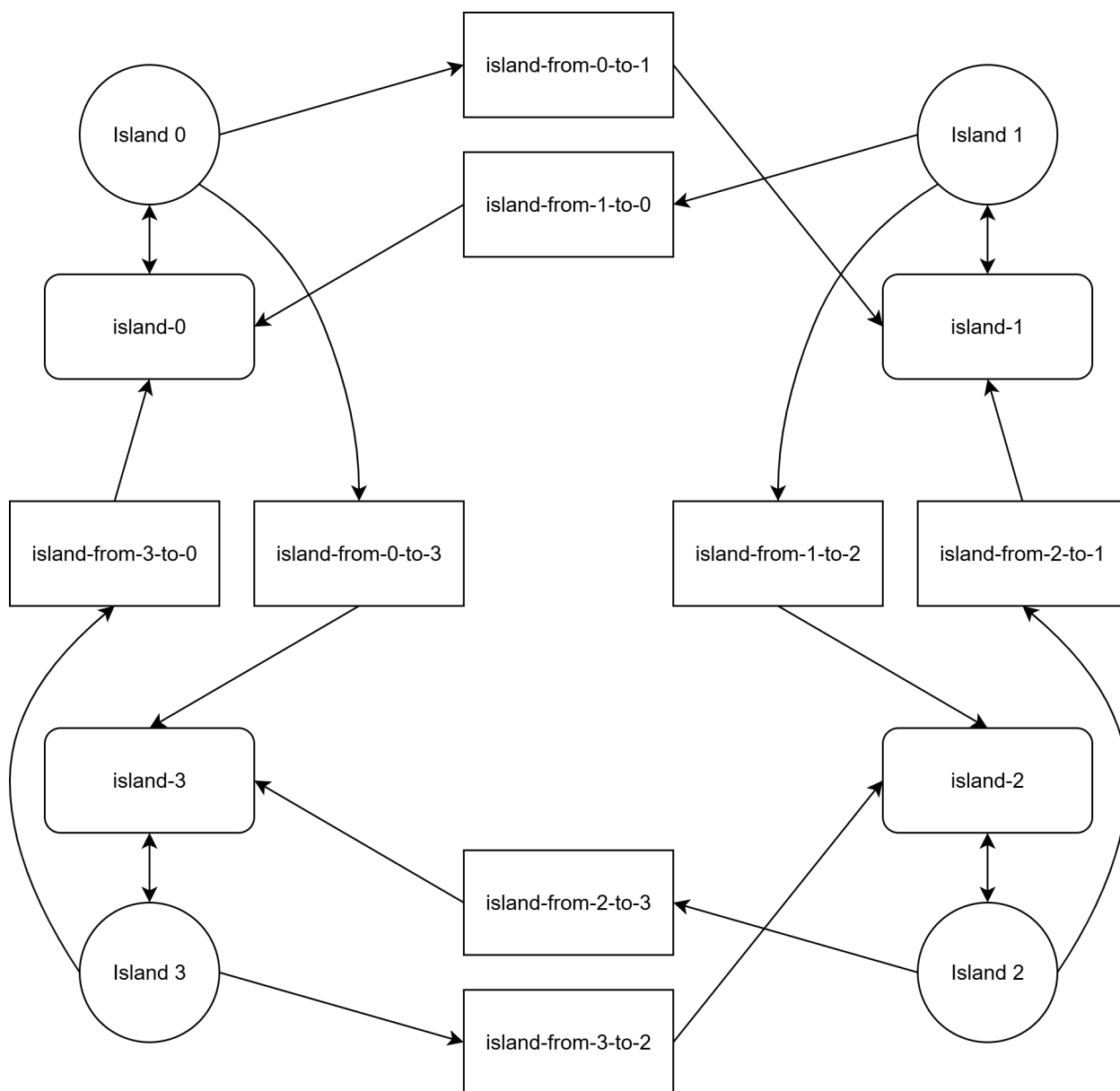
Kod źródłowy 5.12.: Fragment pliku konfiguracyjnego algorytmu w formacie json (odpowiada architekturze z rysunku 5.3)

```
1  "island_delays": {  
2      "0": [-1, 5, -1, 5],  
3      "1": [5, -1, 5, -1],  
4      "2": [-1, 5, -1, 5],  
5      "3": [5, -1, 5, -1]  
6  }
```

Klucze od „0” do „3” reprezentują wyspy–nadawców, a listy będące odpowiadającymi im wartościami przedstawiają wyspy–odbiorców. Kolejki tworzone są za pomocą metody *create\_queues* (kod źródłowy 5.13). Widać tam, że indeksy listy odpowiadają wyspom, a ich wartości opóźnieniom (gdy wartość jest nieujemna) lub brakowi komunikacji (gdy wartość jest równa -1). Przykładowo drugi wiersz fragmentu konfiguracji 5.12 oznacza, że z wyspy „0” agenci mogą emigrować na wyspy „1” i „3” z dodatkowym opóźnieniem wynoszącym 5ms. Nie mogą natomiast zostać wysłani na pozostałe wyspy.

Kod źródłowy 5.13.: Skrócony kod źródłowy służący do tworzenia kolejek w RabbitMQ na podstawie konfiguracji

```
1 connection = pika.BlockingConnection(connection_params)
2 channel = connection.channel()
3 delay_channel = connection.channel()
4 rabbitmq_delays = configuration["island_delays"]
5
6 def create_queues():
7     for island in range(number_of_islands):
8         queue_name = f"island-{island}"
9         channel.queue_declare(queue=queue_name)
10        for i in range(number_of_islands):
11            if i != island:
12                delay = rabbitmq_delays[str(island)][i]
13                if delay == -1:
14                    continue
15                try:
16                    delay_channel.queue_declare(
17                        queue=f"island-from-{island}-to-{i}",
18                        arguments={
19                            "x-message-ttl": delay,
20                            "x-dead-letter-exchange": "amq.direct",
21                            "x-dead-letter-routing-key": f"island-{i}",
22                        },
23                    )
24                except IndexError as e:
25                    exit(1)
26 connection.close()
```



Rysunek 5.3.: Schemat architektury w RabbitMQ dla topologii pierścieniowej z opóźnieniem 5ms

Na schemacie 5.3 widać 3 rodzaje węzłów:

1. okrąg – symbolizuje wyspę
2. prostokąt – reprezentuje kolejkę, do której wyspa–nadawca wysyła emigrantów
3. prostokąt z zaokrąglonymi kątami – przedstawia kolejkę, z której odpowiadająca jej wyspa–odbiorca odczytuje imigrantów

W jaki sposób w takim razie wprowadzone zostaje wybrane dodatkowe opóźnienie? Wiersze 16–23 kodu źródłowego 5.3 przypisują kolejki o kluczach "island-from-x-to-y" (prostokąty) do innego kanału niż kolejki "island-z" (prostokąty z zaokrąglonymi kątami) przypisując niedomyślne argumenty:

1. *"x-message-ttl": delay* – czas życia wiadomości w kolejce (TTL) w milisekundach
2. *"x-dead-letter-exchange": "amq.direct"* – typ wymiany dead-letter-exchange do jakiej przesłana zostanie wiadomość po przekroczeniu jej TTL (tutaj: direct exchange – opisane w rozdziale 4.4)
3. *"x-dead-letter-routing-key": f"island-i"* – klucz routingu wiadomości wysyłanej do dead-letter-exchange (oznacza to, że dla kolejki "island-from-x-to-y" dead-letter routing key to "island-y")

Należy dodać, że na schemacie 5.3 pominięte zostały wymiany (exchanges), a krawędzie reprezentują zapytania i przepływ agentów. Można zauważyć, że z kolejek o kluczach "island-from-x-to-y" (prostokąty) nikt nie próbuje nic odczytać. Oznacza to, że każda wiadomość, która tam trafi z odpowiadającej jej wyspy, umrze po przekroczeniu swojego TTL i dopiero wówczas będzie przesłana do kolejki typu "island-z" po krawędzi widocznej na rysunku. Dokładnie na tym etapie dodawane jest opóźnienie. Wyspa (okrąg) wysyła zapytania do kolejki "island-z", gdzie *z* to numer tej wyspy. Robi to za pomocą metody z wycinku kodu 5.14. Parametr *auto\_ack=True* zapewnia automatyczne potwierdzenie otrzymania wiadomości, co znacznie ogranicza ruch sieciowy i poprawia dzięki temu efektywność rozwiązania.

Kod źródłowy 5.14.: Wycinek kodu źródłowego, za pomocą którego wyspa pobiera imigrantów z kolejki

```
1 method, properties, body = channel.basic_get(f"island-{i}", auto_ack=True)
```

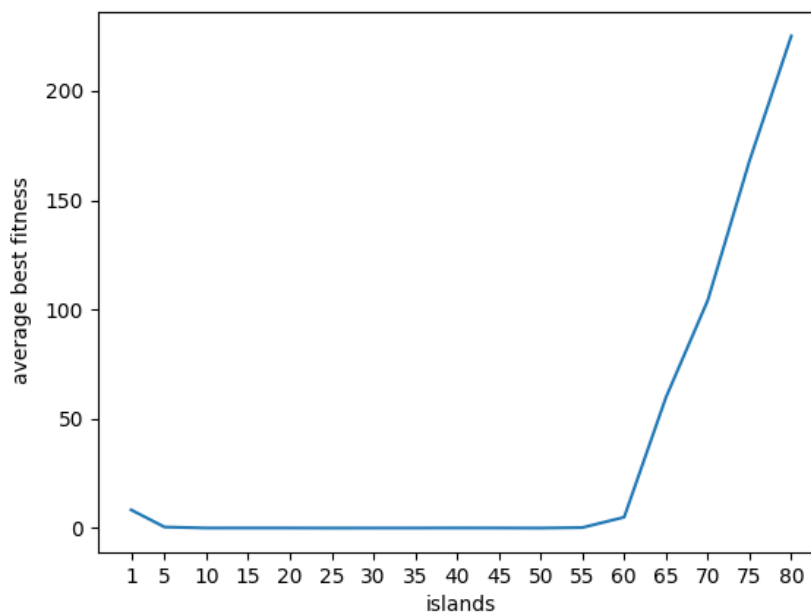


## 6. Badania eksperymentalne

### 6.1. Ograniczenia

Zwiększając ilość komunikacji w obliczeniach problemem staje się szybkość ich przetwarzania przez system kolejkowy.

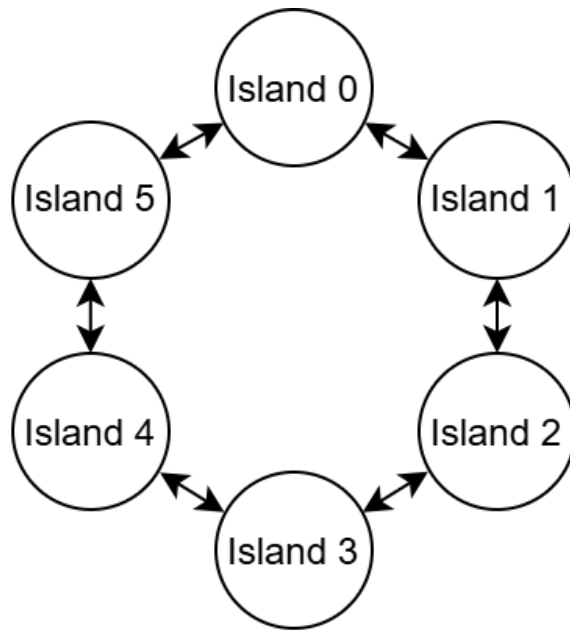
Na wykresie 6.1 widać średnią najlepszą uzyskaną wartość fitness na wyspach w zależności od ich ilości. Można zauważyć, że wynik znacznie polepsza się przy wprowadzeniu migracji (w przypadku jednej wyspy nie ma migracji). Wykorzystując więcej niż 55 wysp rezultaty pogarszają się. Przy 60 wyspach wynik ( $avg\_fitness_{60} = 4.98312$ ) pogorszył się o rząd wielkości w stosunku do 55 wysp ( $avg\_fitness_{55} = 0.252623$ ). Powyżej 60 wysp wynik staje się gorszy niż bez migracji.



Rysunek 6.1.: Średnia najlepsza uzyskana wartość fitness w zależności od ilości wysp

Powyższe obliczenia wykonano używając następującej konfiguracji:

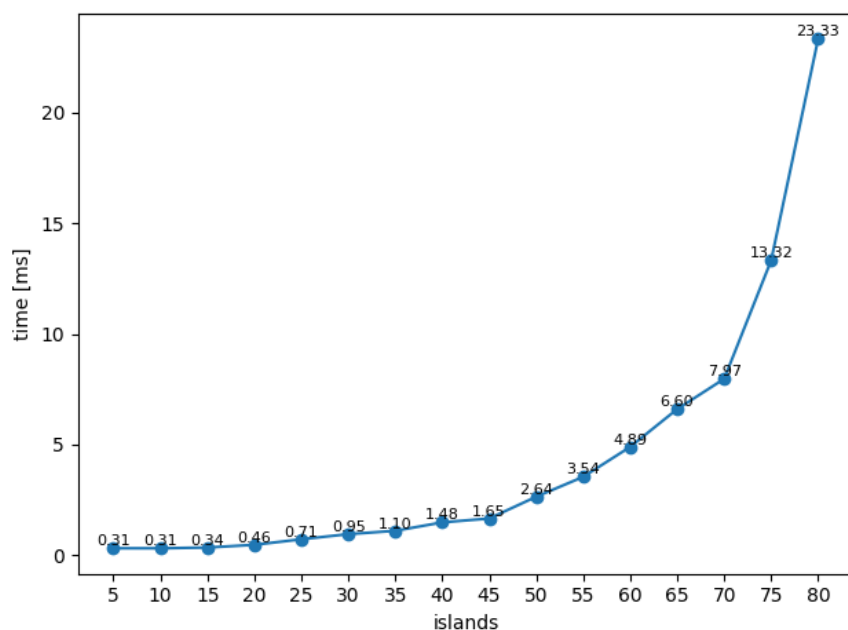
- nodes = ilość wysp (dla Ray) + 2 (dla RabbitMQ);
- tasks per node = 1;
- cpus per task = 16;
- memory per node = 64GB;
- czas obliczeń = 15s;
- opóźnienie = 10ms;
- migracje:
  - topologia pierścieniowa (grafika 6.2)
  - co 16 ewaluacji każda wyspa wysyła po dwóch migrantów do obu sąsiednich wysp
  - komunikacja nieblokująca



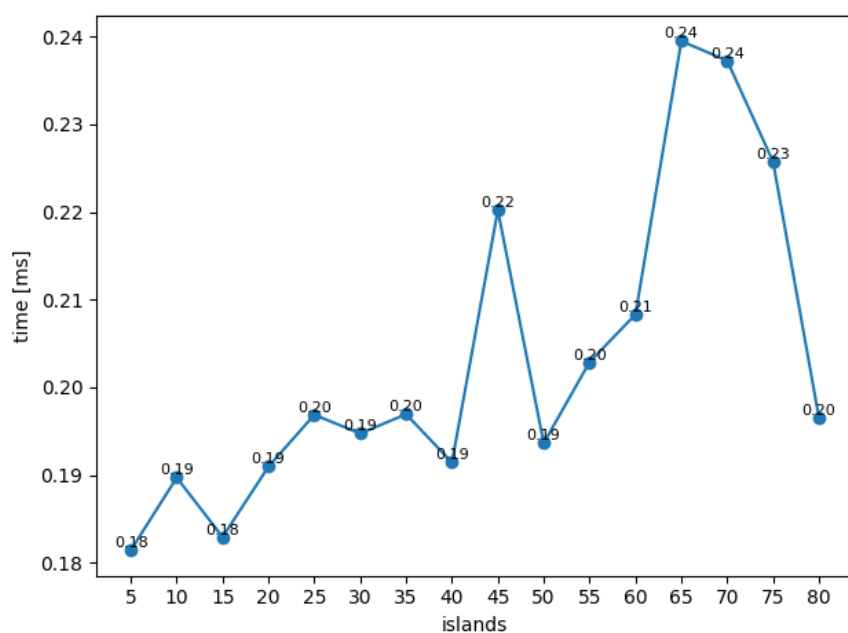
Rysunek 6.2.: Schemat wysp w topologii pierścieniowej

Przy takich ustawieniach wyróżniają się dwie metryki zapisywane podczas obliczeń. Są to:

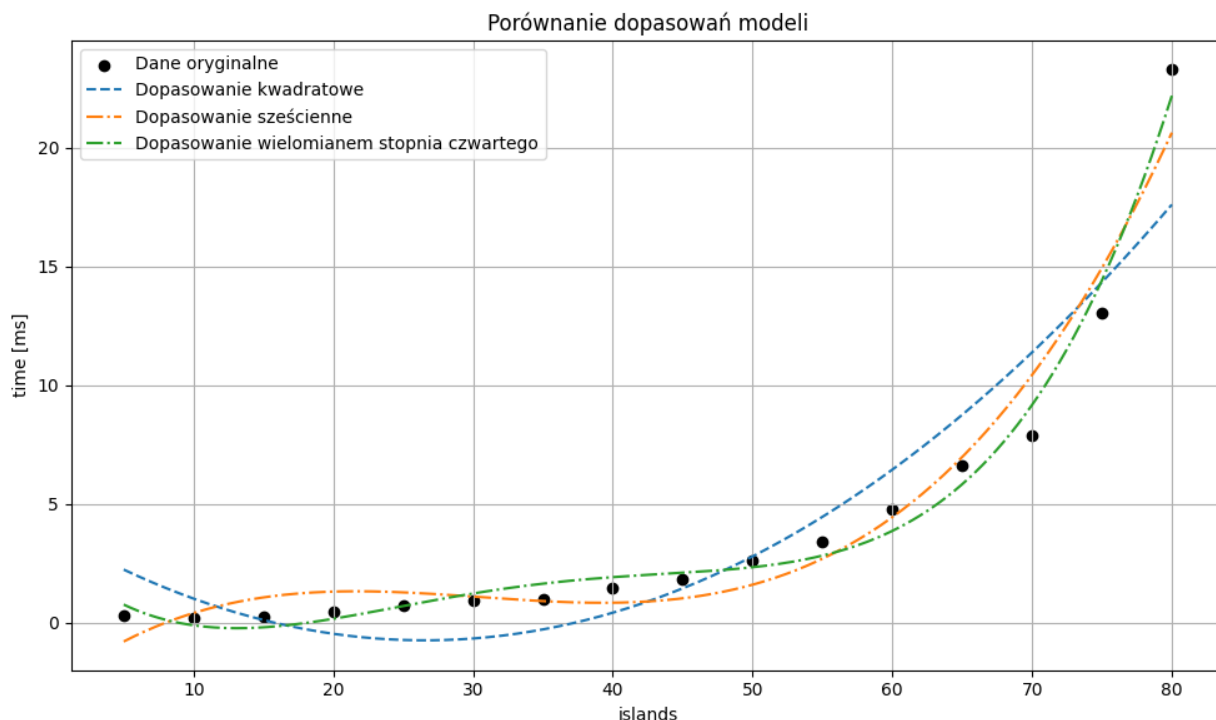
- średni czas pobierania imigranta z kolejki (wykres 6.3)
- średni czas trwania zapytania kolejki o imigranta, kiedy ta jest pusta (wykres 6.4)



Rysunek 6.3.: Średni czas pobierania osobnika z kolejki w zależności od ilości wysp



Rysunek 6.4.: Średni czas próby pobrania osobnika z kolejki, gdy ta jest pusta w zależności od ilości wysp



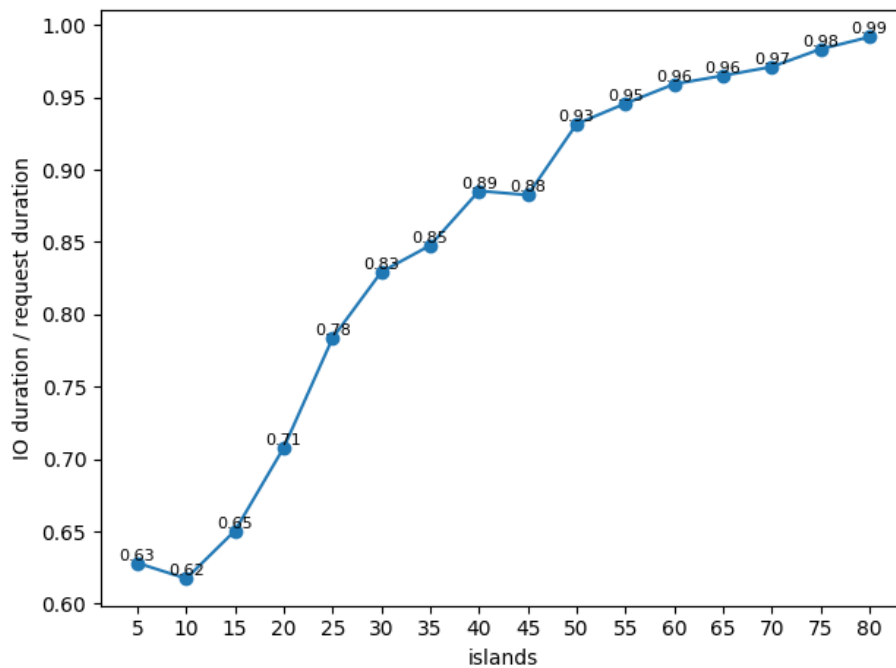
Rysunek 6.5.: Porównanie dopasowań funkcji wielomianowych do czasów pobierania imigrantów w zależności od ilości wysp

Średni czas pobierania imigranta z kolejki rośnie wielomianowo (wykres 6.5) wraz ze wzrostem ilości wykorzystywanych wysp. Podobnej charakterystyki nie widać jednak na wykresie średnich czasów zapytań kolejki, gdy jest ona pusta. Dlaczego tak się dzieje? Odpowiedź na to pytanie wyjaśni, dlaczego od pewnej ilości użytych w obliczeniach wysp jakość rozwiązania gwałtownie się pogarsza.

RabbitMQ zapisuje wiadomości na dysku. Operacja pobrania imigranta z kolejki odbywa się w następujący sposób:

1. klient wysyła zapytanie do kolejki o przesłanie nowego imigranta
2. kolejka wysyła odpowiedź:
  - a) w przypadku, gdy kolejka jest pusta – odpowiedź bez nowego osobnika
  - b) w przeciwnym wypadku – odczytuje wiadomość z danymi osobnika z dysku i przesyła ją jako odpowiedź

Mając ten proces na uwadze, z wykresów 6.3 i 6.4 wynika, że ograniczeniem skalowalności jest zdolność kolejki do szybkiego przetwarzania operacji IO. Udział czasu trwania operacji IO w czasie trwania całego zapytania w zależności od ilości wysp obliczyć można wyznaczając różnicę między odpowiadającymi sobie wartościami na obu wykresach. Stosunek czasu poświęconego na operacje IO do czasu obsługi całego zapytania widoczny jest na wykresie 6.6. Wynika z niego, że wraz z rosnącą liczbą wykorzystywanych wysp (a więc także rosnącą ilością wiadomości), operacje IO zajmują proporcjonalnie coraz większą część czasu w zapytaniu.



Rysunek 6.6.: Średni stosunek czasu spędzonego na wykonywanie operacji IO do czasu obsługi całego zapytania w zależności od ilości wysp

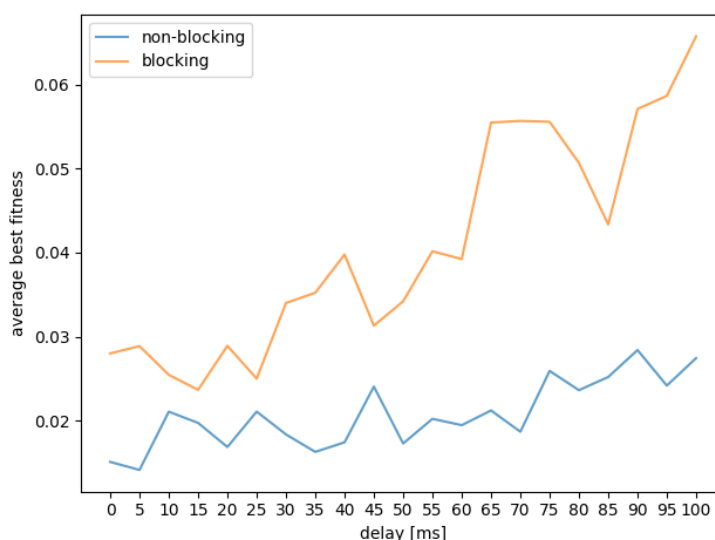
Ograniczenie to zostało wzięte pod uwagę prowadząc kolejne badania. Ilość przesyłanych komunikatów uwzględnia zdolności przetwarzania operacji IO przez system kolejkowy.

## 6.2. Porównanie algorytmu zdesynchronizowanego i synchronicznego

W celu oceny skuteczności wprowadzenia desynchronizacji do algorytmu EMAS przeprowadzono pomiary dwóch jego wersji: zdesynchronizowanego i synchronicznego. Schemat działania algorytmu zdesynchronizowanego został przedstawiony w rozdziale 3.4.

Obliczenia wykonano używając następującej konfiguracji:

- nodes = 27 (25 wysp + 2 dla RabbitMQ);
- tasks per node = 1;
- cpus per task = 16;
- memory per node = 64GB;
- czas obliczeń = 15s;
- migracje:
  - topologia pierścieniowa (grafika 6.2)
  - co 64 ewaluacje każda wyspa wysyła po dwóch migrantów do obu sąsiednich wysp

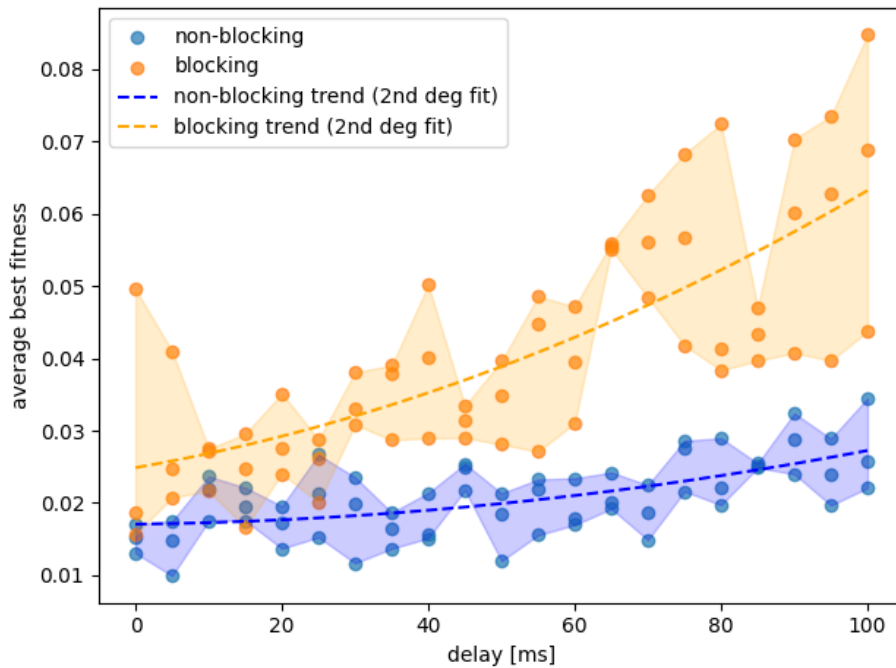


Rysunek 6.7.: Porównanie średnich najlepszych wyników algorytmu zdesynchronizowanego i synchronicznego w zależności od opóźnienia

Na wykresie 6.7 przedstawione zostały średnie najlepszych wyników algorytmu w wersji zdesynchronizowanej (podpisane *non-blocking*) oraz synchronicznej (*blocking*) w zależności od opóźnienia. Przeprowadzone zostały po trzy pomiary na wariant. Algorytm w wersji nieblokującej za każdym razem uzyskał lepszy średni wynik. Średnie wyniki nie rosną monotonicznie przy rosnących opóźnieniach. Metaheurystyki optymalizacyjne charakteryzują się losowością i na jej skutek pewną nieprzewidywalnością wyniku.

Analiza niezagregowanych pomiarów (wykres 6.8) pokazuje, że przy niskich opóźnieniach (do 25ms), mimo gorszej średniej poszczególne wyniki algorytmu synchronicznego mogą być lepsze niż zdesynchronizowanego.

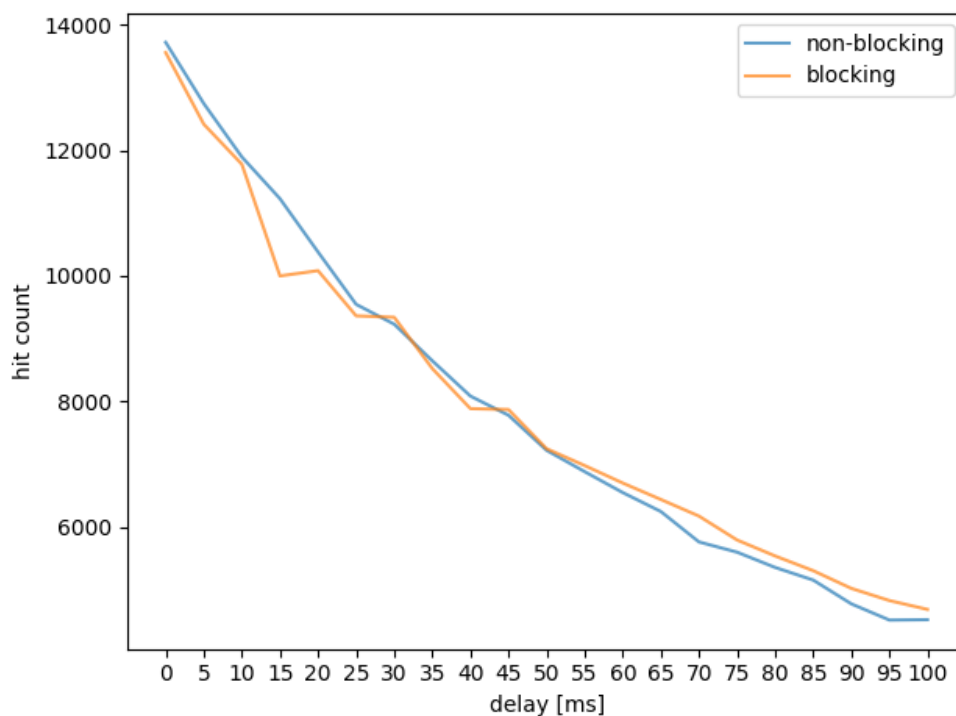
Przerywaną linią na wykresie zaznaczono dopasowanie wielomianem drugiego stopnia średnich wyników pomiarów dla obu wariantów algorytmu. Przy rosnących opóźnieniach rezultaty algorytmu blokującego pogarszają się szybciej.



Rysunek 6.8.: Porównanie najlepszych wyników algorytmu zdesynchronizowanego i synchronicznego w zależności od opóźnienia z dopasowaniem trendu średnich

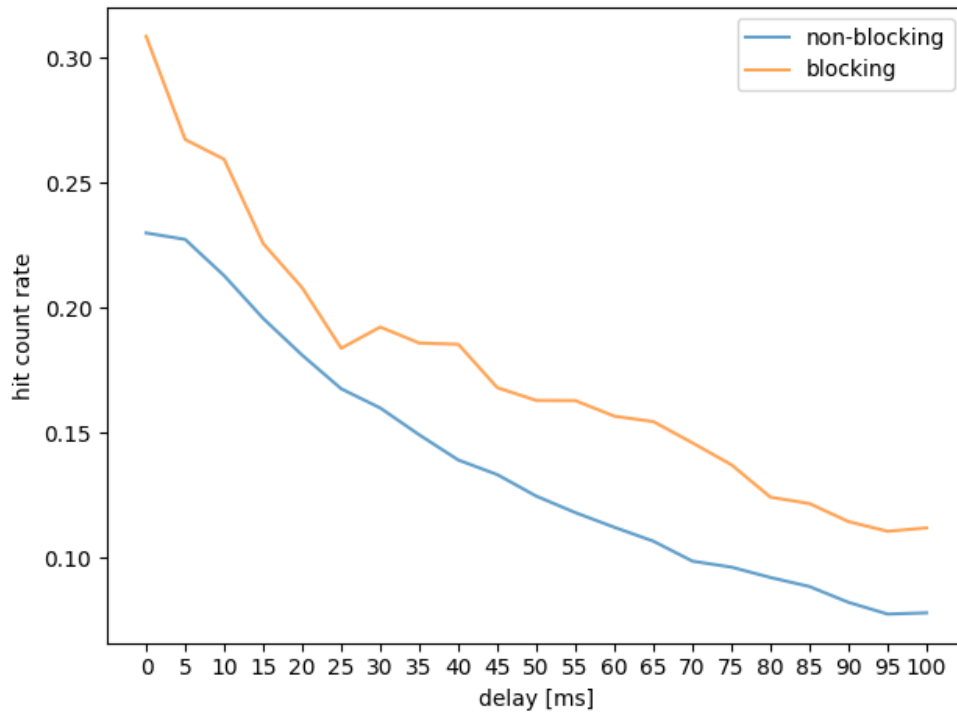
„Trafienie” (*hit*) – nowy osobnik, który poprawił najlepszy wynik wyspy na skutek migracji.

Wykres 6.9 przedstawia porównanie średnich ilości trafień algorytmu zdesynchronizowanego i synchronicznego w zależności od opóźnienia. Dla obu wersji algorytmu wartości są bardzo zbliżone.



Rysunek 6.9.: Porównanie średnich ilości trafień algorytmu zdesynchronizowanego i synchronicznego w zależności od opóźnienia

Różnica pomiędzy obiema wersjami jest widoczna porównując średnie stosunki ilości trafień do ilości przyjętych imigrantów (wykres 6.10).



Rysunek 6.10.: Porównanie średnich stosunków ilości trafień do ilości przybyłych imigrantów algorytmu zdesynchronizowanego i synchronicznego w zależności od opóźnienia

Ponieważ wyniki wariantu zdesynchronizowanego są lepsze, stosunek trafień do liczby imigrantów niższy, a liczba trafień w obu wersjach podobna, można wnioskować, że nawet jeśli nowy osobnik nie poprawia bezpośrednio wyniku wyspy, to jego obecność umożliwia jej pracę nad nowymi, potencjalnie lepszymi rozwiązaniami bez konieczności czekania na przybycie pozostałych agentów.

Powyższa idea jest wykorzystywana w trzech następnych rozdziałach. W 6.3 porównywane są różne metody wyboru wyspy, na którą agent wyemigruje, w 6.4 badane są strategie decydowania o tym, czy należy zaakceptować imigranta i przyjąć go na wyspę, natomiast w 6.5 sprawdzane są sposoby selekcji agenta z populacji do emigracji.

### 6.3. Porównanie metod selekcji wysp do emigracji

W badaniach w rozdziałach 6.1 oraz 6.2 wykorzystywana była następująca konfiguracja emigracji: co ustaloną liczbę ewaluacji na wyspie do dwóch sąsiednich wysp w topologii pierścieniowej wysyłanych było po dwóch osobników o najlepszej wartości fitness. Platforma „Islands Desync” pozwala zmienić topologię migracji przykładowo na kompletną („każdy z każdym” – grafika 3.2). Ponadto wprowadzenie desynchronizacji umożliwia wykorzystanie różnych metod wyboru wyspy docelowej dla osobnika. Dzieje się tak, ponieważ w komunikacji blokującej nigdy nie wiadomo, czy należy czekać na nowego imigranta, jeśli nie ma gwarancji, że ma się on pojawić na wyspie.

W tym rozdziale badane będą następujące metody wyboru wyspy docelowej dla osobników:

- RandomSelect – wybierana jest losowa wyspa
- MaxFitnessSelect – wyspa z najgorszym wynikiem wybierana jest jako docelowa (wynik wyspy, to najniższa wartość fitness agentów na wyspie)
- MinFitnessSelect – wyspa z najlepszym wynikiem wybierana jest jako docelowa
- MinStdDevSelect – wybierana jest wyspa o najniższej różnorodności agentów
- MaxDistanceSelect – wybierana jest wyspa, na której znajduje się agent, który jest najróżniejszy od emigrującego osobnika (najróżniejszy, czyli taki, którego suma kwadratów różnic wartości kolejnych genów jest najwyższa)

Dodatkowo ich wyniki zostaną zestawione z migracją agentów do sąsiednich wysp w topologii pierścieniowej jako punkt odniesienia – SendEverywhere.

Na etapie testów zaimplementowanych metod okazało się, że powyższe strategie (poza RandomSelect i SendEverywhere) działają nieefektywnie, gdy wszyscy emigrujący agenci wysłani zostają na tę samą wyznaczoną przez wyliczoną metrykę wyspę.

Dla zobrazowania na uproszczonym przykładzie z wykorzystaniem MaxFitnessSelect:

1. wyspa *A* ma w kroku migracyjnym 1 najgorszy wynik i aktualizuje swój stan dla sąsiadów
2. wszystkie wyspy wysyłają swoich najlepszych agentów na wyspę *A*
3. wyspa *A* otrzymuje wielu imigrantów (z czego duża ich część może być do siebie podobna)
4. krok ewolucyjny na wyspie *A* trwa dłużej przez obecność znacznie większej liczby agentów niż na pozostałych wyspach

5. w kroku migracyjnym 2 wyspa *A* nadal ma najgorszy wynik, bo w międzyczasie pozostałe wyspy poprawiły same swój wynik szybciej niż *A*
6. wyspa *A* ma najgorszy wynik, więc dostaje ponownie wszystkich agentów, a pozostałe wyspy nie otrzymują żadnych imigrantów, więc działają jak obliczane na jednym procesorze, osiągając słaby wynik końcowy

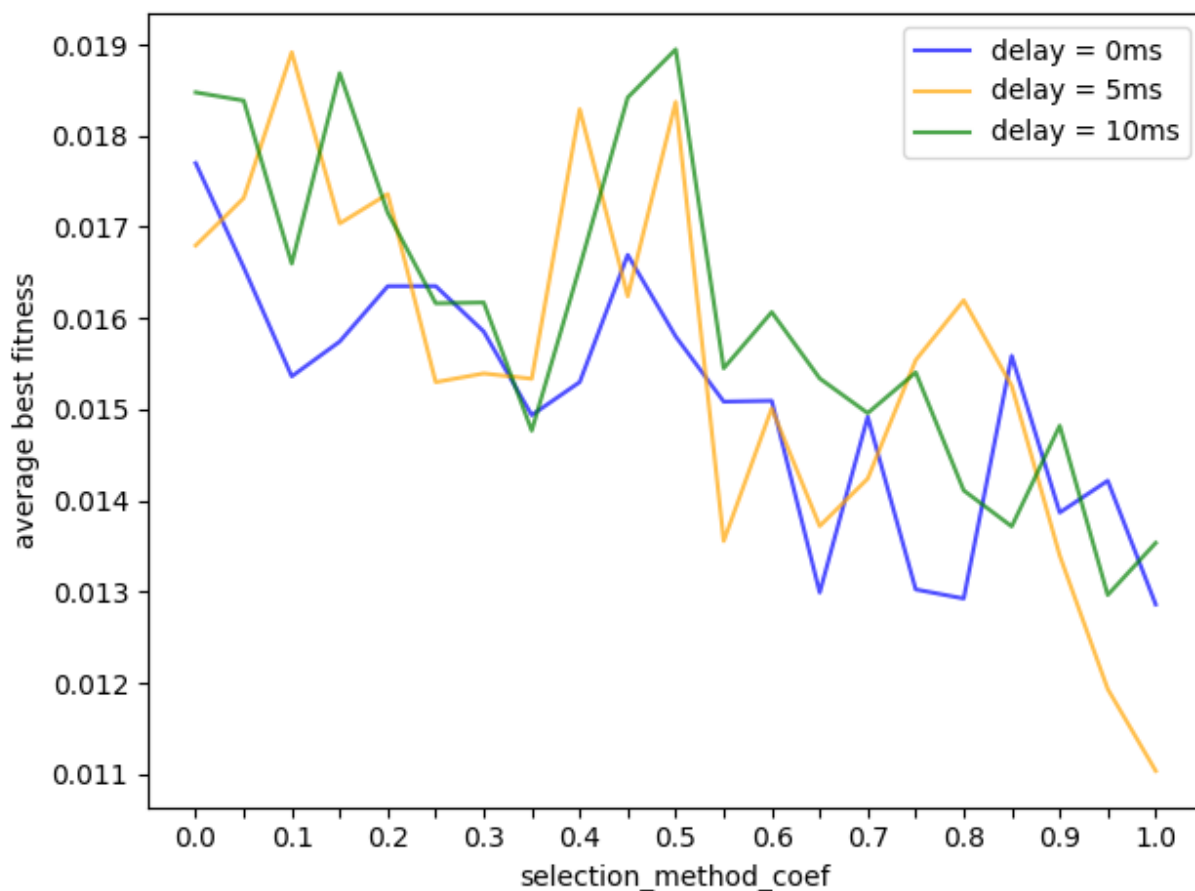
Niech wyspa wykonuje średnio 2000 iteracji na sekundę ( $it\_rate = 2000/s$ ), a migracja następuje średnio co 10 iteracji ( $it\_interval = 10$ ). Wówczas odstęp czasowy między krokami migracyjnymi zajmuje  $it\_interval * \frac{1}{it\_rate} = 10 * \frac{1}{2000/s} = \frac{s}{200} = 5ms$ . Jeśli opóźnienie między wysłaniem, a otrzymaniem agenta trwa  $5ms$ , to jest szansa, że jedna wyspa otrzyma nowych osobników od pozostałych wysp dwukrotnie, zanim zarejestrują one, że wyspa docelowa poprawiła swój wynik, co prowadzi do scenariusza opisanego wyżej.

Aby przeciwdziałać temu zjawisku zastosowane zostały dwa rozwiązania:

1. tylko jeden agent wysyłany jest do wyznaczonej na podstawie strategii emigracyjnej wyspy (pozostałe osobniki wytypowane do migracji wysyłane są na wyspę wylosowaną za pomocą metody RandomSelect)
2. nie w każdym kroku migracyjnym zachodzi sytuacja z punktu 1. – odbywa się to z prawdopodobieństwem  $selection\_method\_coef \in [0, 1]$  – w przeciwnym wypadku każdy agent wysyłany jest zgodnie z metodą RandomSelect

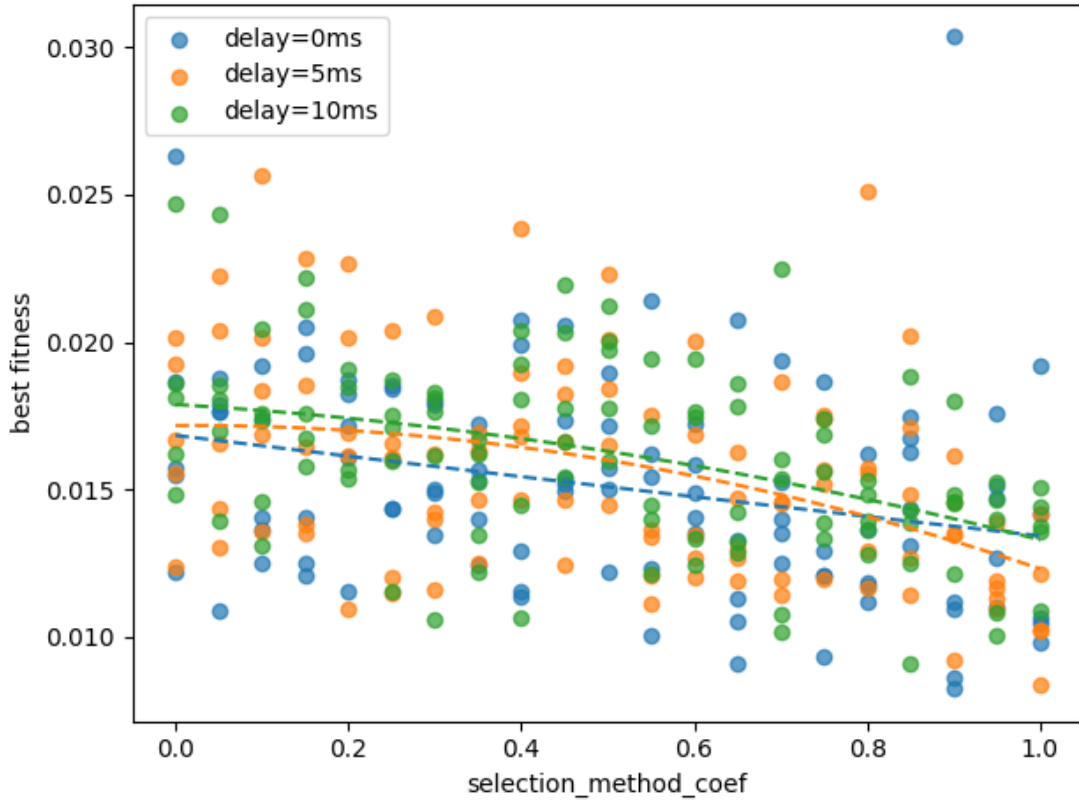
Efekt opisanych powyżej rozwiązań przedstawiony jest na wykresie 6.11. Obliczenia wykonane zostały w następującej konfiguracji:

- nodes = 27 (25 wysp + 2 dla RabbitMQ);
- tasks per node = 1;
- cpus per task = 16;
- memory per node = 64GB;
- czas obliczeń = 15s;
- migracje:
  - topologia pełna (grafika 3.2)
  - co 64 ewaluacje każda wyspa wysyła jednego agenta do wyspy wyznaczonej strategią MaxFitnessSelect i trzech do wysp wskazanych za pomocą metody RandomSelect
  - komunikacja nieblokująca



Rysunek 6.11.: Porównanie średnich najlepszych wyników obliczeń wykorzystujących strategię MaxFitnessSelect w zależności od opóźnienia i współczynnika  $selection\_method\_coef \in [0, 1]$

Na grafie 6.11 widać dużą zmienność wyników przy małej zmianie współczynnika  $selection\_method\_coef$ . Da się jednak zauważyć, że przy rosnących jego wartościach średnie najlepszych wyników spadają. Na wykresie 6.12 przedstawione są poszczególne wyniki na podstawie, których wyznaczone zostały średnie oraz ich linie trendu.



Rysunek 6.12.: Porównanie najlepszych wyników obliczeń wykorzystujących strategię MaxFitnessSelect w zależności od opóźnienia i współczynnika  $selection\_method\_coef \in [0, 1]$  z liniami trendu ich średnich

Wyniki nie są jednoznaczne, jednak linie trendu zaznaczone przerywanymi liniami wskazują na poprawę średnich wyników, gdy  $selection\_method\_coef \in [0.75, 1]$ . Testy wykazały, że także strategie MinFitnessSelect, MaxDistanceSelect, MinStdDevSelect cechują się podobną charakterystyką wyników w zależności od  $selection\_method\_coef$  i z tego powodu w następnych rozdziałach testy przeprowadzone zostały tylko w tym zakresie.

### 6.3.1. Strategia SendEverywhere

Metoda SendEverywhere była już używana w obliczeniach w poprzednich rozdziałach. Dla uzyskania dokładniejszych wyników przeprowadzone zostało więcej pomiarów, których wyniki przedstawione są w tabeli 6.1.

Tabela 6.1.: Wyniki pomiarów działania strategii SendEverywhere oraz ich średnia w zależności od opóźnienia

| Pomiar  | Opóźnienie<br>0 ms | Opóźnienie<br>5 ms | Opóźnienie<br>10 ms |
|---------|--------------------|--------------------|---------------------|
| 1       | 0.01417            | 0.01532            | 0.01739             |
| 2       | 0.00981            | 0.01299            | 0.01214             |
| 3       | 0.01699            | 0.02011            | 0.01334             |
| 4       | 0.01807            | 0.01297            | 0.01735             |
| 5       | 0.01257            | 0.01285            | 0.01032             |
| 6       | 0.01140            | 0.01643            | 0.01606             |
| 7       | 0.01120            | 0.00970            | 0.01563             |
| 8       | 0.01257            | 0.01445            | 0.01914             |
| 9       | 0.01037            | 0.01768            | 0.01825             |
| 10      | 0.01083            | 0.01488            | 0.01287             |
| Średnia | 0.01280            | 0.01474            | 0.01525             |

### 6.3.2. Strategia RandomSelect

RandomSelect jest prostą metodą, która pozwala wykorzystać topologie charakteryzujące się dużą liczbą sąsiadów. Wysyłanie agentów do wszystkich przyległych w takich topologiach wysp powoduje gorsze wyniki niż przy braku migracji. Dzieje się tak na dwa sposoby:

- przy niskim interwale migracji: wyspy otrzymują dużo imigrantów, więc kroki ewolucyjne z ich wykorzystaniem stają się dłuższe i efektywnie testowanych jest mniej nowych rozwiązań oraz migracja zajmuje więcej czasu, a otrzymywani agenci często są podobni do siebie przez co ich większa ilość nie przynosi zysku
- przy wysokim interwale migracji: wyspy rzadziej dzielą się swoimi rozwiązaniami, więc dobre rozwiązania nie mogą być w międzyczasie wykorzystywane na innych wyspach w celu poprawienia swojego wyniku

RandomSelect pozwala przy niskim interwale migracji otrzymywać odpowiednio małą liczbę imigrantów. Dodatkowo metoda jest bardzo prosta i nie wymaga wymiany informacji między wyspami o ich stanie (co jest potrzebne do wyznaczania wysp docelowych w innych strategiach).

Wyniki pomiarów metody przedstawione są w tabeli [6.2](#).

Tabela 6.2.: Najniższe wartości fitness obliczeń z wykorzystaniem strategii RandomSelect oraz ich średnia w zależności od opóźnienia

| Pomiar  | Opóźnienie<br>0 ms | Opóźnienie<br>5 ms | Opóźnienie<br>10 ms |
|---------|--------------------|--------------------|---------------------|
| 1       | 0.01220            | 0.00974            | 0.01702             |
| 2       | 0.01113            | 0.01022            | 0.01423             |
| 3       | 0.01004            | 0.01280            | 0.01486             |
| 4       | 0.01673            | 0.01134            | 0.01860             |
| 5       | 0.01031            | 0.00922            | 0.01977             |
| 6       | 0.01585            | 0.02192            | 0.01224             |
| 7       | 0.01191            | 0.01588            | 0.01530             |
| 8       | 0.01322            | 0.01309            | 0.01068             |
| 9       | 0.01514            | 0.01204            | 0.01182             |
| 10      | 0.01123            | 0.01801            | 0.01451             |
| Średnia | 0.01278            | 0.01343            | 0.01490             |

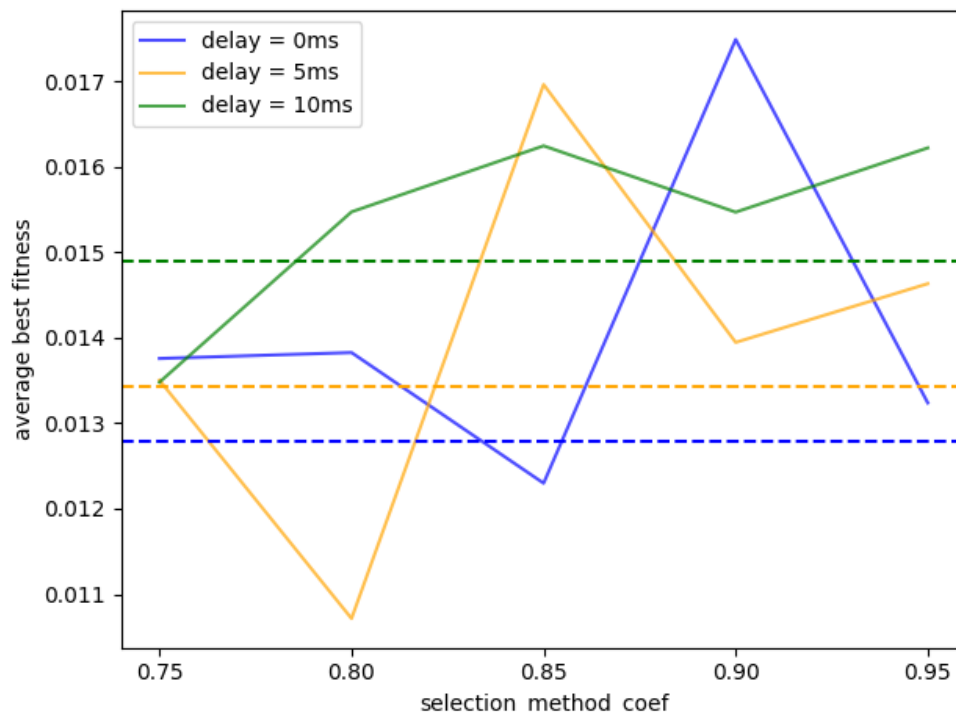
Tabela 6.3 zestawia ze sobą wyniki metod SendEverywhere i RandomSelect. Wynika z niej, że ta druga uzyskała lepsze średnie wyniki. Zmiana topologii oraz strategii migracji pozwoliła poprawić rezultaty algorytmu. Wyniki RandomSelect zostały wykorzystane testując pozostałe strategie dla porównania.

Tabela 6.3.: Porównanie średnich najlepszych wyników metod SendEverywhere i RandomSelect w zależności od opóźnienia

|                           | Opóźnienie<br>0 ms | Opóźnienie<br>5 ms | Opóźnienie<br>10 ms |
|---------------------------|--------------------|--------------------|---------------------|
| Średnia<br>SendEverywhere | 0.01280            | 0.01474            | 0.01525             |
| Średnia<br>RandomSelect   | 0.01278            | 0.01343            | 0.01490             |

### 6.3.3. Strategia MaxFitnessSelect

Ideą stojącą za metodą MaxFitnessSelect jest próba „popchnięcia” do przodu wysp z najgorszymi wynikami. Dzięki migracji nowych agentów z wysp z lepszymi wynikami wyspa docelowa będzie mogła poznać genotypy osobników o niższej wartości fitness i przez krzyżowanie ich z tymi obecnymi na wyspie wytworzyć nowe, potencjalnie lepsze rozwiązania.

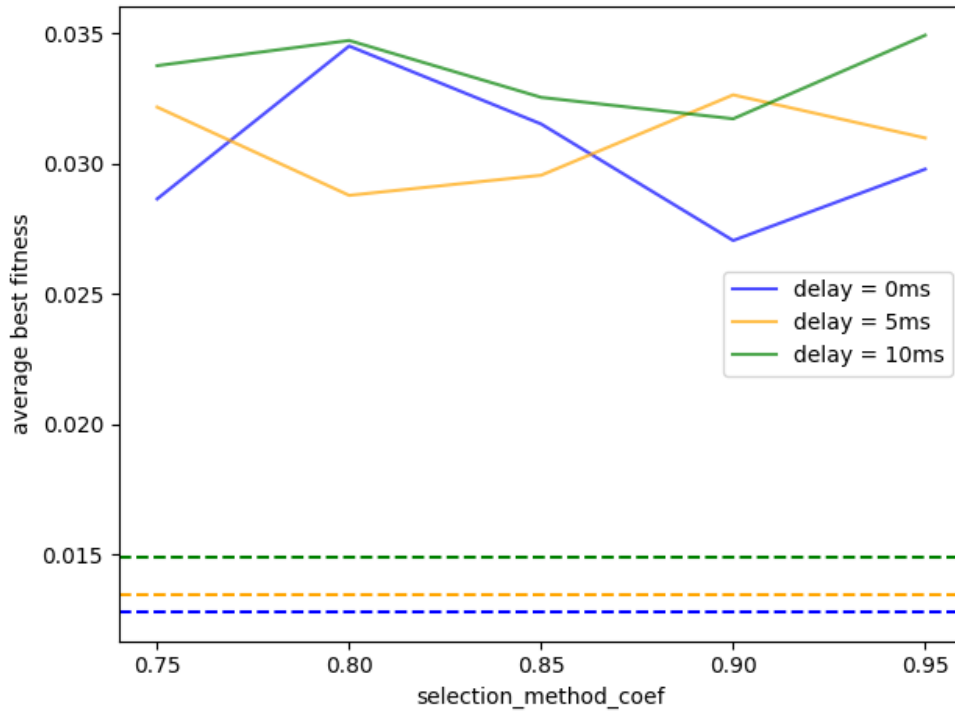


Rysunek 6.13.: Porównanie średnich najlepszych wyników obliczeń wykorzystujących strategię MaxFitnessSelect w zależności od opóźnienia i współczynnika  $selection\_method\_coef \in [0.75, 1]$  w zestawieniu z RandomSelect

Wykres 6.13 przedstawia porównanie średnich najlepszych wyników obliczeń wykorzystujących strategię MaxFitnessSelect w zależności od opóźnienia i  $selection\_method\_coef \in [0.75, 1]$ . Wykonane zostało po 5 pomiarów na wariant. Niestety podobnie jak w przypadku wykresu 6.12 wyniki nie są jednoznaczne i nie wynika z nich przewaga którejś konkretnej konfiguracji. Można jednak zauważyć, że, gdy  $selection\_method\_coef = 1.0$  wyniki są zbliżone do średnich z pomiarów z tabeli 6.2 i osiągnęły względnie dobry rezultat.

#### 6.3.4. Strategia MinFitnessSelect

Podczas gdy strategia MaxFitnessSelect ma pomagać słabszym wyspom nadgonić w obliczeniach te lepsze, to ideą metody MinFitnessSelect jest zwiększanie różnorodności wyspy z najlepszym wynikiem. Może to jej pomóc nie utykać w minimach lokalnych.

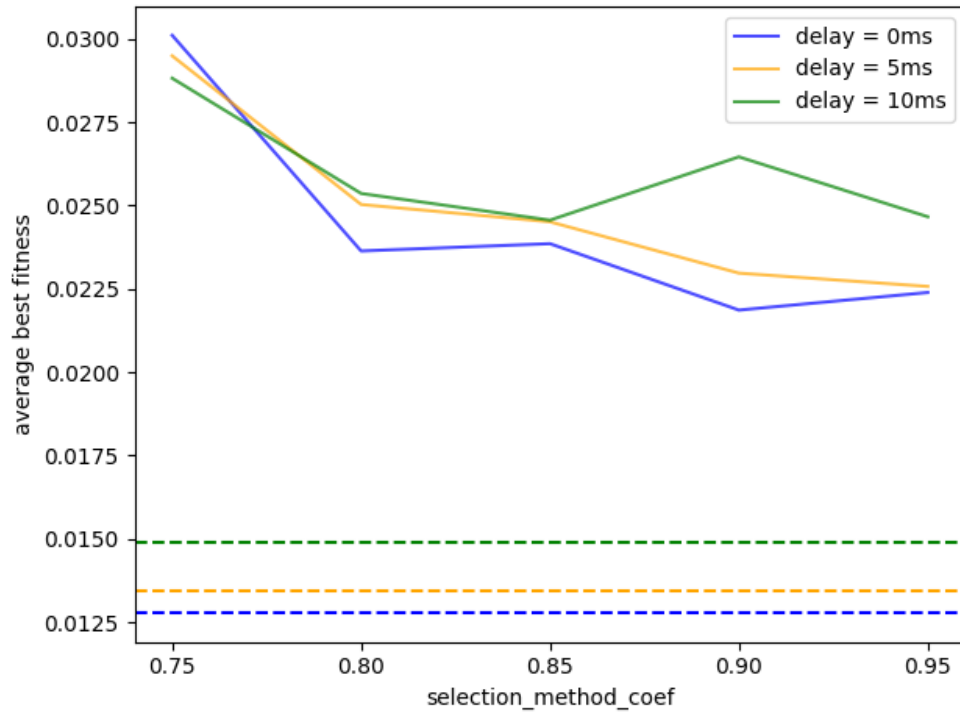


Rysunek 6.14.: Porównanie średnich najlepszych wyników obliczeń wykorzystujących strategię MinFitnessSelect w zależności od opóźnienia i współczynnika  $selection\_method\_coef \in [0.75, 1]$  w zestawieniu z RandomSelect

Metoda niestety nie odniosła oczekiwanych rezultatów.

### 6.3.5. Strategia MinStdDevSelect

Wyspy z niską różnorodnością agentów mogą utykać w minimach lokalnych. Dostarczenie takim wyspom osobników z innych wysp za pomocą metody MinStdDevSelect wprowadza imigrantów z genotypami, które mogą pomóc wyjść z tych minimów i uzyskać nowe, lepsze rozwiązania.

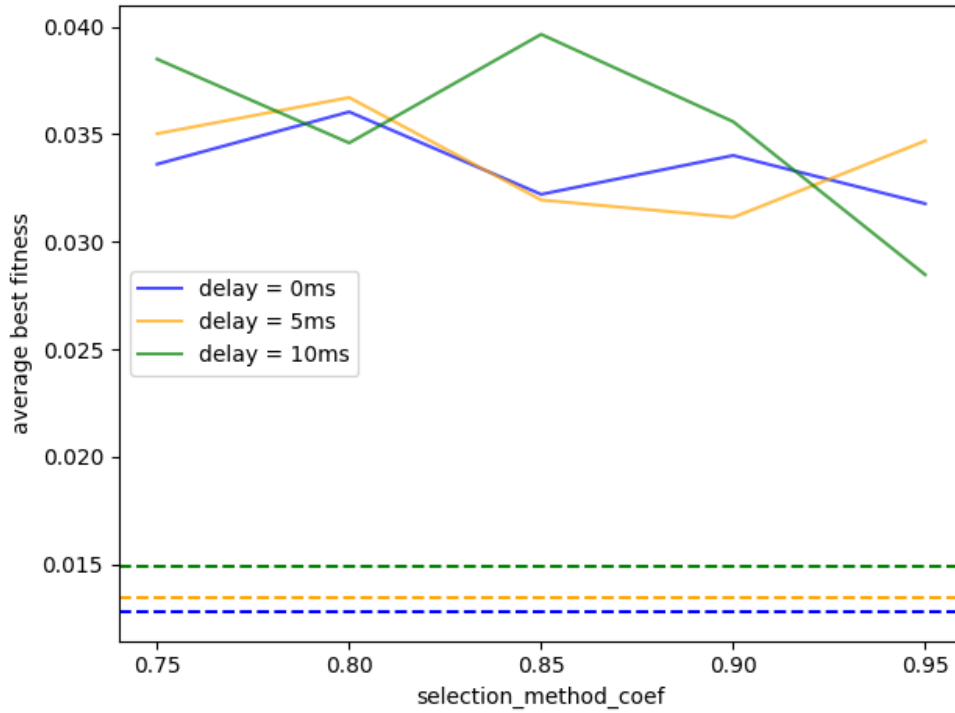


Rysunek 6.15.: Porównanie średnich najlepszych wyników obliczeń wykorzystujących strategię MinStdDevSelect w zależności od opóźnienia i współczynnika  $selection\_method\_coef \in [0.75, 1]$  w zestawieniu z RandomSelect

Mimo, że wyniki są lepsze niż w przypadku poprzedniej metody, to jednak są znacznie gorsze niż RandomSelect.

### 6.3.6. Strategia MaxDistanceSelect

MaxDistanceSelect jest metodą, która jako wyspę docelową wybiera taką, gdzie znajduje się agent najróżniejszy od imigranta, czyli taki, którego wartość  $\sum_{i=0}^{n-1} (migrant.x[i] - agent.x[i])^2$ , gdzie  $migrant.x[i]$  to  $i$ -ty gen imigranta, a  $agent.x[i]$  to odpowiadający mu gen agenta na wyspie, jest największa. Metoda w zamyśle jest podobna do MinStdDevSelect, ale różnorodności szuka na poziomie agenta, a nie wyniku całej wyspy.



Rysunek 6.16.: Porównanie średnich najlepszych wyników obliczeń wykorzystujących strategię MaxDistanceSelect w zależności od opóźnienia i współczynnika  $selection\_method\_coef \in [0.75, 1]$  w zestawieniu z RandomSelect

Metoda niestety nie odniosła oczekiwanych rezultatów.

### 6.3.7. Podsumowanie

Poza MaxFitnessSelect badane strategie nie wykazały zadowalających rezultatów. Ich wyniki były znacząco gorsze niż w przypadku RandomSelect. Może się tak dzieć w związku z faktem, iż metody te wymagają rozsyłania po wyspach dodatkowych informacji. Są to odpowiednio:

- najlepsza wartość fitness na wyspie dla MaxFitnessSelect oraz MinFitnessSelect
- odchylenie standardowe genotypów agentów dla MinStdDevSelect
- genotypy wszystkich agentów obecnych na wyspie dla MaxDistanceSelect

Być może zdolność do podjęcia decyzji o emigracji na podstawie stanu innych wysp nie usprawiedliwia kosztu wymiany tych informacji. Nawet MaxFitnessSelect mimo występujących lepszych wyników niż RandomSelect charakteryzuje się dużą ich zmiennością, co sprawia, że bezpieczniej może być użyć prostszą strategię – RandomSelect. Możliwe jednak że w zależności od konkretnego problemu i wybranej konfiguracji któraś z pozostałych metod poradzi sobie w takim przypadku lepiej.

## 6.4. Porównanie sposobów akceptowania imigrantów

W poprzednim rozdziale badane są strategie migracyjne w kontekście emigracji. Po wyborze wyspy, na którą wysłany zostaje agent, wyspa ta może go przyjąć lub nie. Problem ten został podjęty w tym rozdziale.

Metody, które mogą poprawić efektywność algorytmu to:

- *accept\_everyone* – akceptuj wszystkich imigrantów
- *accept\_above\_avg* – akceptuj tylko imigrantów, których wartość fitness jest lepsza niż średnia wyspy
- *accept\_best* – akceptuj tylko imigrantów, którzy poprawiają wynik wyspy

Dodatkowo w każdej z metod nie są przyjmowani agenci, których genotyp występuje już na wyspie. Może się tak przykładowo zdarzyć, gdy wyspa *A* wyśle osobnika  $\alpha$  do wyspy *B*, a w kolejnym kroku migracyjnym *A* i *B* wyślą  $\alpha$  na wyspę *C* (zwłaszcza, że w kroku migracyjnym może być wysyłanych wielu agentów). Dwóch agentów o takim samym genotypie nie przynosi benefitu wyspie, dlatego warto eliminować kopie.

Ustawienia wykorzystane do badania wpływu eliminacji kopii:

- nodes = 12 (10 wysp + 2 dla RabbitMQ);
- tasks per node = 1;
- cpus per task = 16;
- memory per node = 64GB;
- czas obliczeń = 15s;
- migracje:
  - brak dodatkowego opóźnienia
  - topologia pełna (grafika [3.2](#))
  - co 12 ewaluacji każda wyspa wysyła czterech migrantów na wyspy wybrane za pomocą strategii RandomSelect
  - komunikacja nieblokująca

Tabela 6.4.: Porównanie najniższych wartości fitness uzyskanych bez filtrowania i z eliminacją kopii oraz ich średnie

| Pomiar  | Bez filtrowania | Z eliminacją kopii |
|---------|-----------------|--------------------|
| 1       | 0.10683         | 0.12207            |
| 2       | 0.11497         | 0.09086            |
| 3       | 0.12508         | 0.12527            |
| 4       | 0.08031         | 0.06834            |
| 5       | 0.16284         | 0.12472            |
| 6       | 0.10638         | 0.08264            |
| 7       | 0.14704         | 0.14760            |
| 8       | 0.10216         | 0.12036            |
| 9       | 0.10525         | 0.09748            |
| 10      | 0.14266         | 0.10018            |
| Średnia | 0.11935         | 0.10795            |

Usuwanie kopii przyniosło poprawę otrzymywanych wyników. Konfiguracja pomiarów charakteryzuje się dużą ilością migracji (czterech agentów wysyłanych co dwanaście ewaluacji), aby uwidocznić różnicę jaką daje filtrowanie obecnych już na wyspie genotypów.

Obliczenia przeprowadzone w celu porównania strategii akceptacji imigrantów wykonano używając następującej konfiguracji:

- nodes = 7 (5 wysp + 2 dla RabbitMQ);
- tasks per node = 1;
- cpus per task = 8;
- memory per node = 16GB;
- czas obliczeń = 15s;
- migracje:
  - topologia pełna (grafika 3.2)
  - co 24 ewaluacje każda wyspa wysyła czterech migrantów na wyspy wybrane za pomocą strategii RandomSelect
  - komunikacja nieblokująca

### 6.4.1. Strategia `accept_everyone`

W tabeli 6.5 widać wyniki pomiarów działania algorytmu z wykorzystaniem strategii *accept\_everyone*.

Tabela 6.5.: Najniższe wartości fitness obliczeń z wykorzystaniem metody *accept\_everyone* oraz ich średnia w zależności od opóźnienia

| Pomiar  | Opóźnienie<br>0 ms | Opóźnienie<br>5 ms | Opóźnienie<br>10 ms |
|---------|--------------------|--------------------|---------------------|
| 1       | 0.32756            | 0.38124            | 0.40483             |
| 2       | 0.33010            | 0.36772            | 0.34621             |
| 3       | 0.39820            | 0.48657            | 0.43711             |
| 4       | 0.32047            | 0.39898            | 0.35329             |
| 5       | 0.24996            | 0.42318            | 0.51180             |
| 6       | 0.30372            | 0.38559            | 0.40022             |
| 7       | 0.35396            | 0.30269            | 0.42046             |
| 8       | 0.42622            | 0.42753            | 0.55764             |
| 9       | 0.45627            | 0.38906            | 0.42218             |
| 10      | 0.37618            | 0.29054            | 0.41958             |
| 11      | 0.26805            | 0.35759            | 0.53958             |
| 12      | 0.29792            | 0.32125            | 0.39248             |
| 13      | 0.37969            | 0.54364            | 0.38452             |
| 14      | 0.25325            | 0.42467            | 0.41286             |
| 15      | 0.33738            | 0.38549            | 0.41918             |
| 16      | 0.41848            | 0.40419            | 0.52361             |
| Średnia | 0.34359            | 0.39312            | 0.43410             |

### 6.4.2. Strategia `accept_above_avg`

Najniższe wartości fitness uzyskane podczas obliczeń z użyciem metody *accept\_above\_avg* widoczne są w tabeli 6.6.

Należy napomnieć, że obliczenia w poprzednich rozdziałach wykonane zostały z wykorzystaniem metody *accept\_above\_avg*.

Tabela 6.6.: Najniższe wartości fitness obliczeń z wykorzystaniem metody *accept\_above\_avg* oraz ich średnia w zależności od opóźnienia

| Pomiar  | Opóźnienie<br>0 ms | Opóźnienie<br>5 ms | Opóźnienie<br>10 ms |
|---------|--------------------|--------------------|---------------------|
| 1       | 0.32886            | 0.25806            | 0.34347             |
| 2       | 0.26072            | 0.46597            | 0.45401             |
| 3       | 0.38342            | 0.39273            | 0.45029             |
| 4       | 0.32974            | 0.37421            | 0.53526             |
| 5       | 0.45575            | 0.31003            | 0.67492             |
| 6       | 0.43030            | 0.42968            | 0.46475             |
| 7       | 0.31858            | 0.54403            | 0.21360             |
| 8       | 0.30273            | 0.35223            | 0.44234             |
| 9       | 0.34776            | 0.40266            | 0.34958             |
| 10      | 0.41169            | 0.35525            | 0.36223             |
| 11      | 0.35711            | 0.42403            | 0.42532             |
| 12      | 0.43538            | 0.38726            | 0.31767             |
| 13      | 0.33904            | 0.41031            | 0.46995             |
| 14      | 0.33532            | 0.39392            | 0.32194             |
| 15      | 0.34785            | 0.33998            | 0.53924             |
| 16      | 0.34621            | 0.33223            | 0.50118             |
| Średnia | 0.35815            | 0.38579            | 0.42911             |

### 6.4.3. Strategia *accept\_best*

W tabeli 6.7 przedstawione są wyniki pomiarów działania algorytmu z wykorzystaniem strategii *accept\_best*.

Tabela 6.7.: Najniższe wartości fitness obliczeń z wykorzystaniem metody *accept\_best* oraz ich średnia w zależności od opóźnienia

| Pomiar | Opóźnienie<br>0 ms | Opóźnienie<br>5 ms | Opóźnienie<br>10 ms |
|--------|--------------------|--------------------|---------------------|
| 1      | 0.39077            | 0.40411            | 0.38508             |
| 2      | 0.40569            | 0.45956            | 0.59090             |
| 3      | 0.38087            | 0.37252            | 0.32023             |
| 4      | 0.38173            | 0.33665            | 0.52305             |
| 5      | 0.50426            | 0.48822            | 0.40591             |
| 6      | 0.44952            | 0.34287            | 0.49018             |

| Pomiar  | Opóźnienie<br>0 ms | Opóźnienie<br>5 ms | Opóźnienie<br>10 ms |
|---------|--------------------|--------------------|---------------------|
| 7       | 0.48863            | 0.37132            | 0.53215             |
| 8       | 0.39376            | 0.56509            | 0.45285             |
| 9       | 0.34152            | 0.46762            | 0.39560             |
| 10      | 0.34571            | 0.37703            | 0.44989             |
| 11      | 0.50046            | 0.48668            | 0.44436             |
| 12      | 0.53150            | 0.40779            | 0.41059             |
| 13      | 0.36760            | 0.44548            | 0.70590             |
| 14      | 0.48305            | 0.35789            | 0.60399             |
| 15      | 0.32286            | 0.38773            | 0.37021             |
| 16      | 0.36626            | 0.32585            | 0.37633             |
| Średnia | 0.41589            | 0.41228            | 0.46608             |

#### 6.4.4. Podsumowanie

Średnie otrzymanych wyników w rozdziałach 6.4.1, 6.4.2, 6.4.3 zostały zebrane i zestawione ze sobą w tabeli 6.8.

Tabela 6.8.: Porównanie średnich wyników metod *accept\_everyone*, *accept\_above\_avg* oraz *accept\_best* w zależności od opóźnienia

|                                    | Opóźnienie<br>0 ms | Opóźnienie<br>5 ms | Opóźnienie<br>10 ms |
|------------------------------------|--------------------|--------------------|---------------------|
| Średnia<br><i>accept_everyone</i>  | 0.34359            | 0.39312            | 0.43410             |
| Średnia<br><i>accept_above_avg</i> | 0.35815            | 0.38579            | 0.42911             |
| Średnia<br><i>accept_best</i>      | 0.41589            | 0.41228            | 0.46608             |

Dla każdej testowanej wartości opóźnienia metoda *accept\_best* odniosła najgorsze rezultaty. Przyjmowanie wyłącznie imigrantów poprawiających wynik wyspy sprawdza się gorzej niż dwie pozostałe strategie. Natomiast *accept\_everyone* i *accept\_above\_avg* osiągnęły podobne wyniki. Przy braku dodatkowego opóźnienia metoda *accept\_everyone* zwróciła średnio lepsze rozwiązania, jednak przy wprowadzeniu go wyniki się odwróciły.

Przeprowadzone zostały dodatkowe pomiary (tabela 6.9) dla większego opóźnienia (25ms) i metoda *accept\_everyone* znowu uzyskała lepszy wynik. Na ich podstawie można wysunąć wniosek, że strategia *accept\_above\_avg* sprawdza się lepiej, gdy występuje większe opóźnienie, jednak należy zauważyć, że różnica średnich jest względnie niewielka. Pozostaje także wniosek, że *accept\_everyone* oraz *accept\_above\_avg* są efektywniejsze niż *accept\_best*.

Tabela 6.9.: Najlepsze wyniki i ich średnie metod *accept\_everyone* and *accept\_above\_avg* przy opóźnieniu 25ms

| Pomiar  | <i>accept_everyone</i> | <i>accept_above_avg</i> |
|---------|------------------------|-------------------------|
| 1       | 0.41800                | 0.52974                 |
| 2       | 0.40960                | 0.38222                 |
| 3       | 0.39800                | 0.44928                 |
| 4       | 0.47897                | 0.45632                 |
| 5       | 0.52584                | 0.41479                 |
| 6       | 0.53322                | 0.46685                 |
| 7       | 0.42052                | 0.41992                 |
| 8       | 0.54458                | 0.51947                 |
| 9       | 0.39084                | 0.46394                 |
| 10      | 0.43526                | 0.53500                 |
| 11      | 0.46642                | 0.45401                 |
| 12      | 0.40437                | 0.53080                 |
| 13      | 0.40006                | 0.38049                 |
| 14      | 0.53159                | 0.39609                 |
| 15      | 0.42581                | 0.38231                 |
| 16      | 0.48215                | 0.42924                 |
| Średnia | 0.45408                | 0.45065                 |

## 6.5. Porównanie sposobów wyboru emigrantów

W rozdziałach 6.3 i 6.4 przebadane zostały kolejno strategie selekcji wysp do emigracji oraz sposobów akceptowania imigrantów. Mechanizmem, który może poprawić wyniki algorytmu jest wybór odpowiednich agentów jako emigrantów. Dotychczas wysyłane były osobniki o najniższych wartościach fitness.

Metody, które mogą poprawić efektywność algorytmu to:

- *best* – selekcja agenta o najniższej wartości fitness
- *random* – wybór losowego osobnika
- *maxDistance* – selekcja najróżniejszego agenta w stosunku do pozostałych obecnych na wyspie
- *worst* – wybór osobnika o najgorszej wartości fitness

Powyższe metody przetestowane zostały używając następujących parametrów:

- nodes = 12 (10 wysp + 2 dla RabbitMQ);
- tasks per node = 1;
- cpus per task = 16;
- memory per node = 64GB;
- czas obliczeń = 15s;
- migracje:
  - topologia pełna (grafika 3.2)
  - co 32 ewaluacje każda wyspa wysyła czterech migrantów na wyspy wybrane za pomocą strategii RandomSelect i przyjmowane z wykorzystaniem metody *accept\_everyone*
  - komunikacja nieblokująca

Wyniki pomiarów przedstawione są w rozdziałach 6.5.1, 6.5.2, 6.5.3 oraz 6.5.4, a ich podsumowanie w 6.5.5.

### 6.5.1. Strategia best

Tabela 6.10 przedstawia najlepsze uzyskane wartości fitness w pomiarach z wykorzystaniem strategii *best* w zależności od opóźnienia oraz średnie z przeprowadzonych pomiarów.

Tabela 6.10.: Najniższe wartości fitness obliczeń z wykorzystaniem metody *best* oraz ich średnia w zależności od opóźnienia

| Pomiar  | Opóźnienie 0ms | Opóźnienie 5ms | Opóźnienie 10ms |
|---------|----------------|----------------|-----------------|
| 1       | 0.08648        | 0.09088        | 0.07392         |
| 2       | 0.08381        | 0.08273        | 0.09596         |
| 3       | 0.09390        | 0.10192        | 0.11246         |
| 4       | 0.07031        | 0.11929        | 0.08042         |
| 5       | 0.07967        | 0.09419        | 0.06311         |
| 6       | 0.08352        | 0.07260        | 0.08326         |
| 7       | 0.09385        | 0.08312        | 0.06774         |
| 8       | 0.07255        | 0.07814        | 0.12856         |
| 9       | 0.09284        | 0.06661        | 0.11378         |
| 10      | 0.09269        | 0.08614        | 0.10799         |
| Średnia | 0.08496        | 0.08756        | 0.09272         |

### 6.5.2. Strategia random

Wyniki pomiarów metody *random* widoczne są w tabeli 6.11.

Tabela 6.11.: Najniższe wartości fitness obliczeń z wykorzystaniem metody *random* oraz ich średnia w zależności od opóźnienia

| Pomiar  | Opóźnienie 0ms | Opóźnienie 5ms | Opóźnienie 10ms |
|---------|----------------|----------------|-----------------|
| 1       | 0.11273        | 0.08430        | 0.08827         |
| 2       | 0.09850        | 0.10838        | 0.12936         |
| 3       | 0.07326        | 0.10473        | 0.10034         |
| 4       | 0.10005        | 0.10765        | 0.10420         |
| 5       | 0.09591        | 0.10598        | 0.10746         |
| 6       | 0.10749        | 0.08833        | 0.08583         |
| 7       | 0.08998        | 0.11493        | 0.07637         |
| 8       | 0.12765        | 0.08856        | 0.08541         |
| 9       | 0.08567        | 0.09215        | 0.13219         |
| 10      | 0.07653        | 0.08985        | 0.11640         |
| Średnia | 0.09678        | 0.09849        | 0.10258         |

### 6.5.3. Strategia maxDistance

Najniższe wartości fitness uzyskane podczas obliczeń z użyciem metody *maxDistance* widoczne są w tabeli 6.12.

Tabela 6.12.: Najniższe wartości fitness obliczeń z wykorzystaniem metody *maxDistance* oraz ich średnia w zależności od opóźnienia

| Pomiar  | Opóźnienie 0ms | Opóźnienie 5ms | Opóźnienie 10ms |
|---------|----------------|----------------|-----------------|
| 1       | 0.19317        | 0.32140        | 0.20655         |
| 2       | 0.19727        | 0.24321        | 0.27660         |
| 3       | 0.21998        | 0.16207        | 0.27220         |
| 4       | 0.27567        | 0.27173        | 0.17439         |
| 5       | 0.26581        | 0.22108        | 0.29551         |
| 6       | 0.24391        | 0.26599        | 0.16445         |
| 7       | 0.31674        | 0.28930        | 0.24986         |
| 8       | 0.27985        | 0.23905        | 0.16679         |
| 9       | 0.18905        | 0.17637        | 0.29223         |
| 10      | 0.21340        | 0.27971        | 0.28610         |
| Średnia | 0.23949        | 0.24699        | 0.23847         |

### 6.5.4. Strategia worst

Tabela 6.13 zawiera najlepsze otrzymane wartości fitness oraz ich średnią z wykorzystaniem metody *worst* w zależności od opóźnienia.

Tabela 6.13.: Najniższe wartości fitness obliczeń z wykorzystaniem metody *worst* oraz ich średnia w zależności od opóźnienia

| Pomiar  | Opóźnienie 0ms | Opóźnienie 5ms | Opóźnienie 10ms |
|---------|----------------|----------------|-----------------|
| 1       | 0.15568        | 0.11138        | 0.13111         |
| 2       | 0.11129        | 0.12291        | 0.11098         |
| 3       | 0.11329        | 0.12677        | 0.12382         |
| 4       | 0.10686        | 0.14655        | 0.11050         |
| 5       | 0.13220        | 0.13719        | 0.12175         |
| 6       | 0.15333        | 0.12463        | 0.13835         |
| 7       | 0.12811        | 0.12421        | 0.13496         |
| 8       | 0.09875        | 0.10831        | 0.14959         |
| 9       | 0.17304        | 0.14443        | 0.13519         |
| 10      | 0.11326        | 0.13344        | 0.15020         |
| Średnia | 0.12858        | 0.12798        | 0.13065         |

### 6.5.5. Podsumowanie

Wyniki otrzymane w poprzednich rozdziałach porównane są w tabeli 6.14 poniżej.

Tabela 6.14.: Porównanie średnich najlepszych wyników metod *best*, *random*, *maxDistance* oraz *worst* w zależności od opóźnienia

|                            | Opóźnienie<br>0 ms | Opóźnienie<br>5 ms | Opóźnienie<br>10 ms |
|----------------------------|--------------------|--------------------|---------------------|
| Średnia <i>best</i>        | 0.08496            | 0.08756            | 0.09272             |
| Średnia <i>random</i>      | 0.09678            | 0.09849            | 0.10258             |
| Średnia <i>maxDistance</i> | 0.23949            | 0.24699            | 0.23847             |
| Średnia <i>worst</i>       | 0.12858            | 0.12798            | 0.13065             |

Z powyższego zestawienia wynika, że najskuteczniejsza okazała się metoda *best*. Gorsze, jednak nadal dobre wyniki osiągnęła strategia *random*. Metody *worst* oraz *maxDistance* spisały się najslabiej. Ich rezultaty okazały się znacząco gorsze. Taki wynik pomiarów utrzymuje się niezależnie od wybranego opóźnienia wśród badanych wartości.

## 7. Zakończenie

### 7.1. Podsumowanie i wnioski

Przeprowadzone badania wykazały efektywność zdesynchronizowanego algorytmu EMAS w porównaniu do jego synchronicznej wersji. Co więcej przy rosnącym opóźnieniu w komunikacji jego wyniki pogarszają się wolniej niż w przypadku zachowania komunikacji blokującej.

Przetestowane zostały różne strategie:

1. selekcji wyspy docelowej, na którą wysłany zostanie osobnik
2. podejmowania decyzji o przyjęciu imigranta na wyspę
3. wyboru agenta z populacji do emigracji

W pierwszej kategorii najlepsze wyniki uzyskały metody RandomSelect i MaxFitnessSelect. Pozostałe strategie odniosły znacząco gorsze rezultaty. Losowy wybór wyspy docelowej okazał się zaskakująco skuteczny. Mogłoby się wydawać, że podjęcie decyzji na podstawie dodatkowo przesyłanych informacji o stanach innych wysp może znacząco poprawić efektywność migracji, jednak tak się nie stało. Prostota implementacji, elastyczność przypadków użycia oraz brak konieczności wprowadzania nadmiarowej komunikacji sprawiają, że metoda RandomSelect jest dobrym rozwiązaniem dla szerokiej gamy problemów. Niewykluczone jest jednak, że są scenariusze, w których inne testowane metody poradziłyby sobie lepiej.

Należy zaznaczyć, że wykorzystanie strategii selekcji wyspy docelowej dla emigranta pozwala wykorzystać złożone topologie wysp. W przypadku najprostszej metody polegającej na wysyłaniu osobników do wszystkich sąsiadujących wysp nie można zutilizować przewagi jaką dają środowiska HPC. W konfiguracji, w której wykorzystywane są dziesiątki lub setki węzłów obliczeniowych rozsyłanie migrantów „na ślepo” do każdego z nich, wprowadziłoby ciężki do obsłużenia nakład komunikacyjny i obliczeniowy, który pogarszałby wynik końcowy. Bez sprawdzanych strategii migracji skalowalność algorytmu kończyłaby się na zaledwie kilku wyspach.

W kategorii podejmowania decyzji dotyczącej przyjęcia imigranta na wyspę najlepsze rezultaty uzyskały strategie *accept\_everyone* oraz *accept\_above\_avg*. Gorszy wynik metody *accept\_best* pokazuje, że poszerzenie populacji nie dość, że o agentów niepoprawiających najniższej wartości fitness wyspy, ale nawet o takich poniżej średniej jest skuteczne. Wynika z tego, że samo wprowadzenie nowych genotypów umożliwia algorytmowi EMAS szybsze wyznaczenie nowego poprawionego rozwiązania jako efekt następujących po migracji krzyżowań.

Okazało się także, że eliminacja powtarzających się genotypów w populacji przy okazji procesu imigracji usprawnia działanie algorytmu. Efekty widoczne są szczególnie w sytuacji, gdy ilość imigrantów jest wysoka.

W kwestii wyboru osobnika do emigracji najskuteczniejsza okazała się metoda *best* polegająca na selekcji agentów o najlepszych wartościach fitness. W przeciwieństwie do strategii decydowania o wyspie docelowej dla osobnika użycie informacji o stanie populacji przyniosło wymierną poprawę. Metoda *random* zwróciła gorsze, ale nadal dobre wyniki. Wybieranie najgorszego agenta na wyspie przyniosło lepsze rezultaty niż najróżniejszego od reszty, jednak były one znacznie gorsze niż w przypadku *best* i *random*. Prowadząc obliczenia najlepiej wysyłać agentów o najniższej wartości fitness.

## 7.2. Uwagi

W badaniach wykorzystane zostało środowisko korzystające z systemu kolejkowego RabbitMQ. Zostało ono dodane do platformy „Islands Desync” wyłącznie w celu symulowania opóźnień o wybranej wartości. W rzeczywistym środowisku Ray może samemu zapewnić całość komunikacji między częściami systemu. Przedstawiony w pracy zdesynchronizowany algorytm EMAS może się skalować znacznie lepiej niż testowane w poprzednich rozdziałach konfiguracje, gdyż RabbitMQ stanowi wąskie gardło.

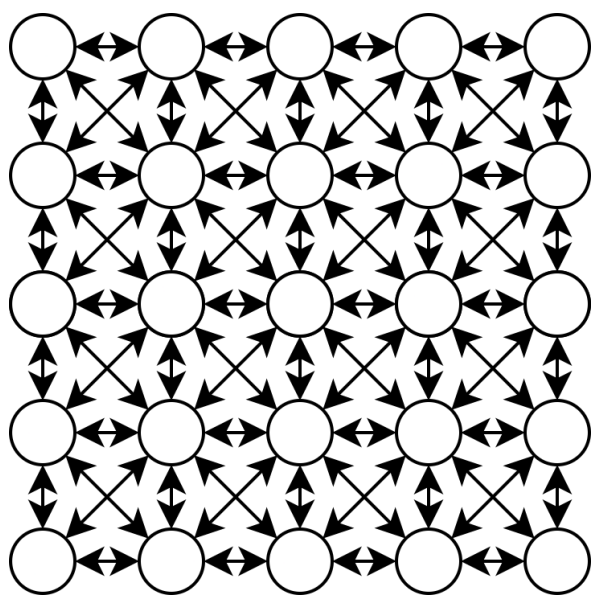
Środowisko badane w pracy można uruchomić w systemie HPC opartym o SLURM zgodnie z instrukcją z dodatku do pracy [7.4](#).

## 7.3. Dalszy rozwój

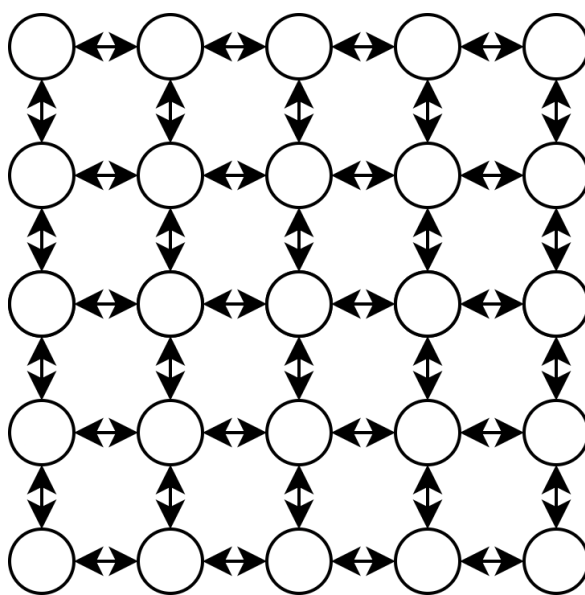
Wykonane zostało wiele pomiarów, jednak ograniczenia przypisanych zasobów na maszynach Cyfronetu nie pozwoliły przeprowadzić ich więcej. Realizacja większej ilości powtórzeń w badanych kategoriach i konfiguracjach umożliwiłaby uzyskanie dokładniejszych i bardziej powtarzalnych rezultatów.

Testowane warianty algorytmu były względnie niezależne. Dostęp do większej ilości zasobów obliczeniowych otworzyłby drzwi do przeprowadzenia bardziej kompleksowych badań. Pozwoliłoby to wówczas wykonać pomiary w formie algorytmu Grid Search wykorzystywanego do tuningu hiperparametrów w dziedzinie sztucznej inteligencji. Możliwe, że pewne parametry, modyfikacje algorytmu, topologie i metody wysyłania oraz przyjmowania osobników na wyspy współgrają w sposób, którego w niezależnych (w znaczeniu, że porównywana jest jedna modyfikacja na raz, a reszta elementów jest niezmienna) badaniach nie da się zauważyć.

Platforma jest rozszerzalna, więc pozytywnie na działanie algorytmu może wpłynąć implementacja nowych, bardziej złożonych topologii (np. siatka 2D z sąsiedztwem Moore'a albo von Neumanna, których schematy widać na rysunkach kolejno 7.1 i 7.2 lub siatka 3D), parametrów zmiennych w czasie działania algorytmu czy alternatywnych strategii emigracji i imigracji.



Rysunek 7.1.: Topologia – siatka 2D z sąsiedztwem Moore'a



Rysunek 7.2.: Topologia – siatka 2D z sąsiedztwem von Neumanna

## 7.4. Dodatek – uruchomienie środowiska

W celu uruchomienia środowiska na systemie HPC zarządzanym za pomocą menadżera zasobów SLURM należy stosować się do poniższej instrukcji.

1. Zainstalować RabbitMQ zgodnie z krokami z dokumentacji [55].
2. Sklonować repozytorium projektu [56] w wybranym folderze.
3. Otworzyć plik *a.sh* w edytorze (przykładowo w *vi*).
4. Wczytać moduł Pythona dostępny na używanym systemie HPC.
5. Zainstalować Pythonowe biblioteki zawarte w skrypcie.
6. Dopasować parametry SLURM z początku pliku (konfiguracja flag po frazie `#SBATCH`) do własnych potrzeb.
7. Zlecić wykonanie skryptu za pomocą komendy *sbatch a.sh*.
8. W przypadku problemów działać zgodnie z pojawiającymi się błędami – konfiguracja może różnić się zależności od konkretnego systemu HPC.

# Bibliografia

- [1] Ashutosh Tiwari i in. „Survey on the use of computational optimisation in UK engineering companies”. W: *CIRP Journal of Manufacturing Science and Technology* 9 (2015), s. 57–68. ISSN: 1755-5817. DOI: <https://doi.org/10.1016/j.cirpj.2015.01.003>. URL: <https://www.sciencedirect.com/science/article/pii/S175558171500005X>.
- [2] *Ranking Top500, listopad 2024*. URL: <https://top500.org/lists/top500/2024/11/>. (dostęp: 11.08.2025).
- [3] *Ranking HPCG, listopad 2024*. URL: <https://top500.org/lists/hpcg/2024/11/>. (dostęp: 11.08.2025).
- [4] *Strona WWW centrum Cyfronet*. URL: <https://www.cyfronet.pl/>. (dostęp: 11.08.2025).
- [5] *Strona WWW platformy PLGrid*. URL: <https://plgrid.pl/>. (dostęp: 11.08.2025).
- [6] *Artykuł opisujący superkomputer Ares*. URL: [https://www.cyfronet.pl/komputery/18486,artykul,superkomputer\\_ares.html](https://www.cyfronet.pl/komputery/18486,artykul,superkomputer_ares.html). (dostęp: 11.08.2025).
- [7] *Ares w rankingu Top500*. URL: <https://www.top500.org/system/179958/>. (dostęp: 11.08.2025).
- [8] *Repozytorium platformy Islands Desync na serwisie GitHub*. URL: [https://github.com/youngdashu/islands\\_desync/](https://github.com/youngdashu/islands_desync/). (dostęp: 11.08.2025).
- [9] Fernando Peres i Mauro Castelli. „Combinatorial Optimization Problems and Metaheuristics: Review, Challenges, Design, and Development”. W: *Applied Sciences* 11.14 (2021). ISSN: 2076-3417. DOI: [10.3390/app11146449](https://doi.org/10.3390/app11146449). URL: <https://www.mdpi.com/2076-3417/11/14/6449>.
- [10] Rajesh Matai, Surya Singh i Murari Lal Mittal. „Traveling Salesman Problem: an Overview of Applications, Formulations, and Solution Approaches”. W: list. 2010. ISBN: 978-953-307-426-9. DOI: [10.5772/12909](https://doi.org/10.5772/12909).
- [11] Maram Assi i Ramzi A. Haraty. „A Survey of the Knapsack Problem”. W: *2018 International Arab Conference on Information Technology (ACIT)*. 2018, s. 1–6. DOI: [10.1109/ACIT.2018.8672677](https://doi.org/10.1109/ACIT.2018.8672677).

- [12] Moatamad R. Hassan. „An efficient method to solve least-cost minimum spanning tree (LC-MST) problem”. W: *Journal of King Saud University - Computer and Information Sciences* 24.2 (2012), s. 101–105. ISSN: 1319-1578. DOI: <https://doi.org/10.1016/j.jksuci.2011.12.001>. URL: <https://www.sciencedirect.com/science/article/pii/S1319157811000401>.
- [13] Marcin Molga i Czesław Smutnicki. „Test functions for optimization needs”. W: *Test functions for optimization needs* 101.48 (2005), s. 32.
- [14] Mohamed Abdel-Basset, Laila Abdel-Fatah i Arun Kumar Sangaiah. „Chapter 10 - Metaheuristic Algorithms: A Comprehensive Review”. W: *Computational Intelligence for Multimedia Big Data on the Cloud with Engineering Applications*. Red. Arun Kumar Sangaiah, Michael Sheng i Zhiyong Zhang. Intelligent Data-Centric Systems. Academic Press, 2018, s. 185–231. ISBN: 978-0-12-813314-9. DOI: <https://doi.org/10.1016/B978-0-12-813314-9.00010-4>. URL: <https://www.sciencedirect.com/science/article/pii/B9780128133149000104>.
- [15] Wissam Marrouche. „Unlocking the Potential of Metaheuristics for NP-Hard Problems”. Prac. dokt. University of Portsmouth, 2024.
- [16] Kenneth Sörensen i Fred W. Glover. „Metaheuristics”. W: *Encyclopedia of Operations Research and Management Science*. Red. Saul I. Gass i Michael C. Fu. Boston, MA: Springer US, 2013, s. 960–970. ISBN: 978-1-4419-1153-7. DOI: [10.1007/978-1-4419-1153-7\\_1167](https://doi.org/10.1007/978-1-4419-1153-7_1167). URL: [https://doi.org/10.1007/978-1-4419-1153-7\\_1167](https://doi.org/10.1007/978-1-4419-1153-7_1167).
- [17] Helena Stegherr, Michael Heider i Jörg Hähner. „Classifying Metaheuristics: Towards a unified multi-level classification system”. W: *Natural Computing* 21 (czer. 2022), s. 1–17. DOI: [10.1007/s11047-020-09824-0](https://doi.org/10.1007/s11047-020-09824-0).
- [18] Pedro Espejo, Sebastian Ventura i Francisco Herrera. „A Survey on the Application of Genetic Programming to Classification”. W: *Systems, Man, and Cybernetics, Part C: Applications and Reviews, IEEE Transactions on* 40 (kw. 2010), s. 121–144. DOI: [10.1109/TSMCC.2009.2033566](https://doi.org/10.1109/TSMCC.2009.2033566).
- [19] Ahmad Hoorfar. „Evolutionary Programming in Electromagnetic Optimization: A Review”. W: *IEEE Transactions on Antennas and Propagation* 55.3 (2007), s. 523–537. DOI: [10.1109/TAP.2007.891306](https://doi.org/10.1109/TAP.2007.891306).
- [20] June Ho Park i in. „Economic load dispatch using evolutionary algorithms”. W: *Proceedings of International Conference on Intelligent System Application to Power Systems*. 1996, s. 441–445.
- [21] Kwang Y. Lee i Frank F. Yang. „Optimal reactive power planning using evolutionary algorithms: a comparative study for evolutionary programming, evolutionary strategy, genetic algorithm, and linear programming”. W: *IEEE Transactions on Power Systems* 13.1 (1998), s. 101–108.

- 
- [22] Annu Lambora, Kunal Gupta i Kriti Chopra. „Genetic Algorithm- A Literature Review”. W: *2019 International Conference on Machine Learning, Big Data, Cloud and Parallel Computing (COMITCon)*. 2019, s. 380–384. DOI: [10.1109/COMITCon.2019.8862255](https://doi.org/10.1109/COMITCon.2019.8862255).
- [23] Tom V. Mathew. „Genetic algorithm”. W: *Report submitted at IIT Bombay* 53 (2012), s. 18–19.
- [24] Enrique Alba, Gabriel Luque i Sergio Nesmachnow. „Parallel metaheuristics: recent advances and new trends”. W: *International Transactions in Operational Research* 20.1 (2013), s. 1–48.
- [25] Lenka Lhotska, Martin Macaš i Miroslav Burša. „PSO and ACO in Optimization Problems”. W: *International conference on intelligent data engineering and automated learning*. Springer. 2006, s. 1390–1398.
- [26] Yudong Zhang, Shuihua Wang i Genlin Ji. „A Comprehensive Survey on Particle Swarm Optimization Algorithm and Its Applications”. W: *Mathematical Problems in Engineering* 2015 (paź. 2015), s. 1–38. DOI: [10.1155/2015/931256](https://doi.org/10.1155/2015/931256).
- [27] Marco Dorigo i Luca Maria Gambardella. „Ant colony system: a cooperative learning approach to the traveling salesman problem”. W: *IEEE Transactions on evolutionary computation* 1.1 (2002), s. 53–66.
- [28] Krzysztof Cetnarowicz, Marek Kisiel-Dorohinicki i Edward Nawarecki. „The application of evolution process in multi-agent world to the prediction system”. W: *Proceedings of the Second International Conference on Multi-Agent Systems, ICMA*. T. 96. 1996, s. 26–32.
- [29] Marek Kisiel-Dorohinicki, Grzegorz Dobrowolski i Edward Nawarecki. „Profile-based architecture of evolutionary MAS”. ENG. W: *TASK Quarterly. Scientific Bulletin of Academic Computer Centre in Gdansk* 7.1 (2003), s. 65–72.
- [30] Marek Kisiel-Dorohinicki. „Agent-Oriented Model of Simulated Evolution”. W: *SO-FSEM 2002: Theory and Practice of Informatics*. Red. William I. Grosky i František Plášil. Berlin, Heidelberg: Springer Berlin Heidelberg, 2002, s. 253–261. ISBN: 978-3-540-36137-4.
- [31] Aleksander Byrski i in. „Evolutionary multi-agent systems”. W: *The Knowledge Engineering Review* 30 (mar. 2015), s. 171–186. DOI: [10.1017/S0269888914000289](https://doi.org/10.1017/S0269888914000289).
- [32] Daniel Krzywicki i in. „Computing agents for decision support systems”. W: *Future Generation Computer Systems* 37 (2014), s. 390–400. ISSN: 0167-739X. DOI: <https://doi.org/10.1016/j.future.2014.02.002>. URL: <https://www.sciencedirect.com/science/article/pii/S0167739X14000272>.
-

- [33] Krzysztof Wróbel i in. „Evolutionary Multi-Agent Computing in Inverse Problems”. W: *Computer Science* 14.3 (czer. 2013), s. 367. DOI: [10.7494/csci.2013.14.3.367](https://doi.org/10.7494/csci.2013.14.3.367). URL: <https://journals.agh.edu.pl/csci/article/view/67>.
- [34] Michał Kowol, Aleksander Byrski i Marek Kisiel-Dorohinicki. „Agent-based Evolutionary Computing for Difficult Discrete Problems”. W: *Procedia Computer Science* 29 (2014), s. 1039–1047. ISSN: 1877-0509. DOI: <https://doi.org/10.1016/j.procs.2014.05.093>. URL: <https://www.sciencedirect.com/science/article/pii/S1877050914002701>.
- [35] Sebastian Pisarski i in. „Evolutionary Multi-Agent System in Hard Benchmark Continuous Optimisation”. W: *Applications of Evolutionary Computation*. Red. Anna I. Esparcia-Alcázar. Berlin, Heidelberg: Springer Berlin Heidelberg, 2013, s. 132–141.
- [36] Mateusz Starzec, Grażyna Starzec i Mateusz Paciorek. „Desynchronization of simulation and optimization algorithms in HPC Environment”. W: *Computer Science* 21 (wrz. 2020). DOI: [10.7494/csci.2020.21.3.3772](https://doi.org/10.7494/csci.2020.21.3.3772).
- [37] Jakub Bujas i in. „High-performance computing framework with desynchronized information propagation for large-scale simulations”. W: *Journal of Computational Science* 32 (2019), s. 70–86. ISSN: 1877-7503. DOI: <https://doi.org/10.1016/j.jocs.2018.09.004>. URL: <https://www.sciencedirect.com/science/article/pii/S1877750318303776>.
- [38] Mateusz Starzec i in. „Desynchronization in distributed Ant Colony Optimization in HPC environment”. W: *Future Generation Computer Systems* 109 (2020), s. 125–133. ISSN: 0167-739X. DOI: <https://doi.org/10.1016/j.future.2020.03.045>. URL: <https://www.sciencedirect.com/science/article/pii/S0167739X19327402>.
- [39] *Strona frameworka MASON*. URL: <https://cs.gmu.edu/~eclab/projects/mason/>. (dostęp: 11.08.2025).
- [40] *Strona frameworka RePast*. URL: <https://repast.github.io/>. (dostęp: 11.08.2025).
- [41] *Strona frameworka MadKit*. URL: <https://www.madkit.net/madkit/>. (dostęp: 11.08.2025).
- [42] *Strona frameworka AgE*. URL: <https://www.age.agh.edu.pl/agent-oriented-platforms/age/>. (dostęp: 11.08.2025).
- [43] Łukasz Faber i in. „Agent-Based Simulation in AgE Framework”. W: t. 416. Sty. 2012, s. 55–83. ISBN: 9783642288876. DOI: [10.1007/978-3-642-28888-3\\_3](https://doi.org/10.1007/978-3-642-28888-3_3).
- [44] Roman Dębski, Aleksander Byrski i Marek Kisiel-Dorohinicki. „Towards an Agent-Based Augmented Cloud”. W: *Journal of Telecommunications and Information Technology* 2012 (mar. 2012). DOI: [10.26636/jtit.2012.1.1248](https://doi.org/10.26636/jtit.2012.1.1248).

- 
- [45] Sylwia Biełaszek i in. „Delays in Computing with Parallel Metaheuristics on HPC Infrastructure”. W: *Computational Collective Intelligence: 16th International Conference, ICCCI 2024, Leipzig, Germany, September 9–11, 2024, Proceedings, Part II*. Leipzig, Germany: Springer-Verlag, 2024, s. 163–175. ISBN: 978-3-031-70818-3. DOI: [10.1007/978-3-031-70819-0\\_13](https://doi.org/10.1007/978-3-031-70819-0_13). URL: [https://doi.org/10.1007/978-3-031-70819-0\\_13](https://doi.org/10.1007/978-3-031-70819-0_13).
- [46] Dokumentacja frameworka Ray. URL: <https://docs.ray.io/en/latest/>. (dostęp: 11.08.2025).
- [47] Philipp Moritz i in. „Ray: A distributed framework for emerging AI applications”. W: *13th USENIX symposium on operating systems design and implementation (OSDI 18)*. 2018, s. 561–577.
- [48] Wykorzystanie SLURM w dokumentacji Cyfronetu. URL: <https://kdm.cyfronet.pl/portal/Podstawy:SLURM>. (dostęp: 11.08.2025).
- [49] Andy B. Yoo, Morris A. Jette i Mark Grondona. „Slurm: Simple linux utility for resource management”. W: *Workshop on job scheduling strategies for parallel processing*. Springer. 2003, s. 44–60.
- [50] Wykorzystanie SLURM w dokumentacji Ray. URL: <https://docs.ray.io/en/latest/cluster/vms/user-guides/community/slurm.html>. (dostęp: 11.08.2025).
- [51] Dokumentacja RabbitMQ. URL: <https://www.rabbitmq.com/docs>. (dostęp: 11.08.2025).
- [52] Valeriu Manuel Ionescu. „The analysis of the performance of RabbitMQ and ActiveMQ”. W: *2015 14th RoEduNet International Conference-Networking in Education and Research (RoEduNet NER)*. IEEE. 2015, s. 132–137.
- [53] John O’Hara. „Toward a commodity enterprise middleware: Can AMQP enable a new era in messaging middleware? A look inside standards-based messaging with AMQP”. W: *Queue* 5.4 (2007), s. 48–55.
- [54] Diego Ongaro i John Ousterhout. „In search of an understandable consensus algorithm”. W: *2014 USENIX annual technical conference (USENIX ATC 14)*. 2014, s. 305–319.
- [55] Dokumentacja procesu instalacji RabbitMQ na podstawie pliku binarnego. URL: <https://www.rabbitmq.com/docs/install-generic-unix>. (dostęp: 11.08.2025).
- [56] Link do repozytorium na Githubie do projektu z kodem części programistycznej pracy magisterskiej. URL: <https://github.com/superiorshrimp/mgr>. (dostęp: 11.08.2025).